

Universitatea Națională de Științe și Tehnologii
POLITEHNICA București

PROIECT 2 – APLICAȚII PRACTICE ALE MICROCONTROLLERELOR

ECHIPA NR. 5

Studenți :

Păun Alexandru-Nicolae

Petrescu Oana Ștefania

Popa Emilia

Șerbănoiu Ana-Maria

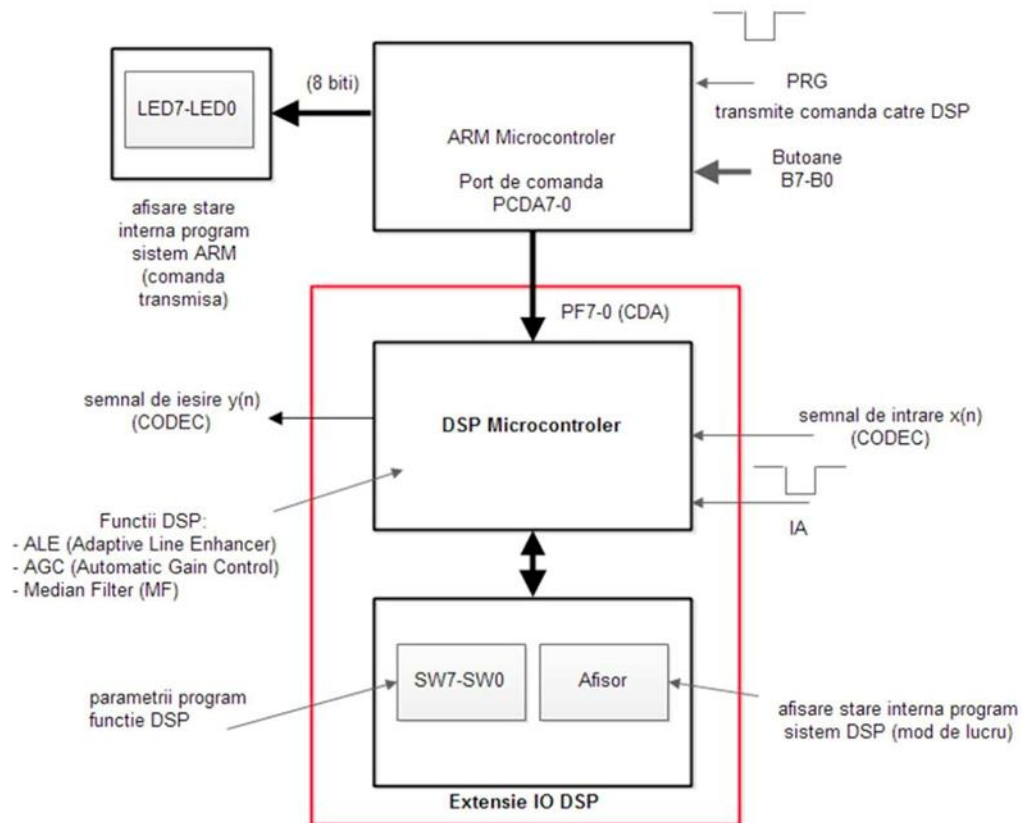
Grupa:

431D

Profesor coordonator : Zoican Sorin

1. Descrierea temei

Sistemul implementat este compus din două subsisteme interconectate — ARM și DSP, având rolul de a procesa semnalele.



Arhitectura sistemului

- **Microcontrolerul ARM (STM32F103CBTx):**

Rolul său este de unitate de comandă. Acesta gestionează interfața cu utilizatorul și transmite comenzile către DSP.

Intrări: Sistemul dispune de 8 butoane (B7-B0) destinate selectării comenzilor, precum și un buton special **PRG** utilizat pentru transmiterea efectivă a comenzii către DSP.

Ieșiri: Un set de 8 LED-uri (LED7-LED0) are rolul de a indica vizual ce comandă a fost trimisă către DSP, oferind feedback imediat utilizatorului.

Interfața de comunicare: Transmiterea comenzii se realizează prin portul **PCDA7-0**, conectat la intrarea **PF7-0** a DSP-ului, asigurând un transfer paralel de date pe 8 biți.

- **Microcontrolerul DSP (ADSP-2181 EZ-KIT Lite):**

Acesta execută algoritmi de prelucrare a semnalului audio în timp real.

Intrări semnal: Primește semnalul analogic prin **CODEC** (intrarea $x(n)$), care reprezintă semnalul audio brut ce urmează să fie procesat.

Ieșiri semnal: Trimite semnalul procesat înapoi prin **CODEC** (ieșirea $y(n)$), rezultatul final al algoritmului de procesare aplicat.

Semnalul IA: Este o intrare de întrerupere asincronă care anunță DSP-ul că ARM-ul a pus o comandă nouă pe port, declanșând astfel rutina de tratare a întreruperii.

Extensia IO DSP: conține comutatoarele SW7-SW0 prin care utilizatorul poate configura manual parametrii funcțiilor DSP, și un afișor care arată starea internă a sistemului DSP în timp real.

Denumirea CODEC reprezintă prescurtarea termenilor COdor și DECodor, cei doi pași fundamentali prin care trece semnalul audio în sistem.

1. Etapa de Codare (ADC - Analog to Digital Converter)

În această etapă, semnalul analogic $x(n)$ este pregătit pentru a putea fi procesat numeric de către DSP:

- **Eșantionarea:** Circuitul măsoară tensiunea semnalului audio la intervale regulate, determinate de frecvența de eșantionare aleasă.
- **Cuantizarea pe 16 biți:** Fiecare măsurătoare este reprezentată ca un număr binar de 16 biți, oferind o rezoluție ridicată și un zgomot de cuantizare redus.
- **Transmisia serială:** Valorile obținute sunt trimise către DSP prin SPORT0 sub forma unui flux de date discrete $x(n)$.

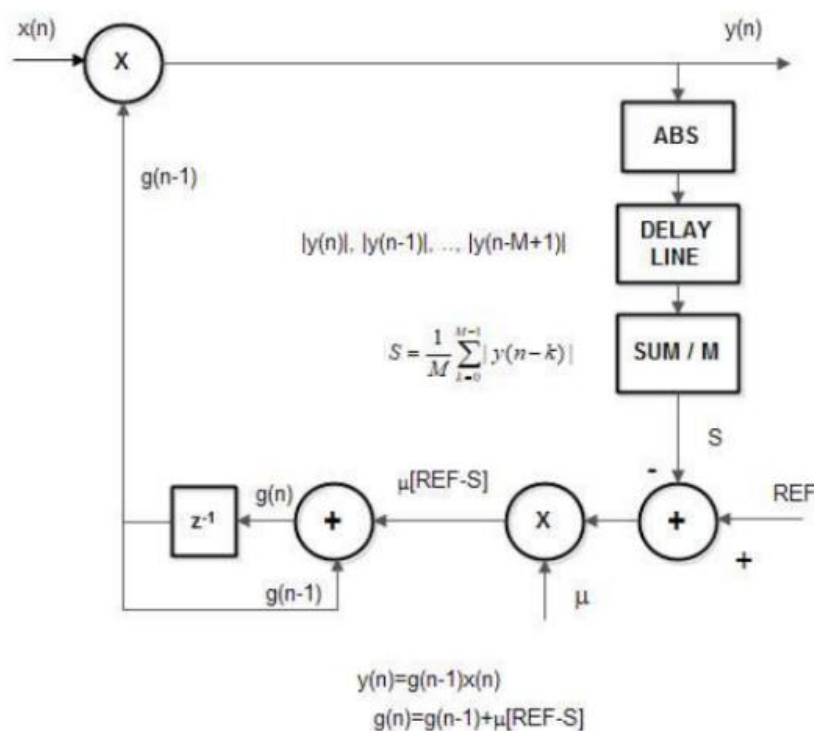
2. Etapa de Decodare (DAC - Digital to Analog Converter)

După ce DSP-ul a aplicat algoritmi de procesare, semnalul $y(n)$ trebuie reconvertit în format analogic:

- **Reconversia numerică:** Codecul primește valorile $y(n)$ calculate de DSP și le transformă în trepte de tensiune proporționale.
- **Netezirea semnalului:** Un filtru intern elimină discontinuitățile dintre trepte, rezultând o undă analogică continuă și curată, gata să fie redată prin difuzor.

2. Descrierea funcțiilor de procesare

- Automatic Gain Control (AGC)



Automatic Gain Control (AGC) - este un algoritm de procesare adaptivă a semnalului audio care ajustează automat amplificarea aplicată semnalului de intrare, astfel încât semnalul de ieșire să rămână la un nivel constant, indiferent dacă sursa este prea slabă sau prea puternică.

Scopul acestui algoritm este de a asigura un volum uniform la ieșire, compensând variațiile de amplitudine ale semnalului de intrare în timp real.

Cum funcționează:

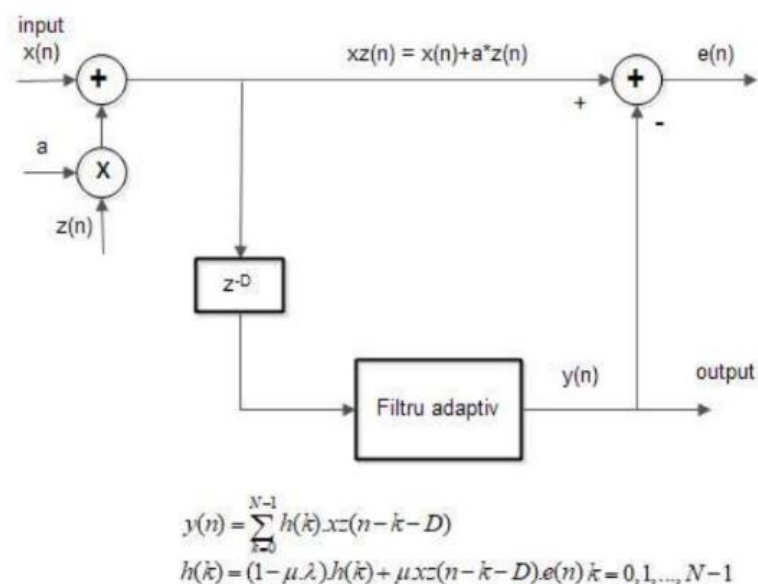
- Sistemul înmulțește fiecare eșantion de intrare $x(n)$ cu un câștig $g(n-1)$ determinat în pasul anterior, obținând ieșirea $y(n) = g(n-1) \cdot x(n)$.
- Se calculează valoarea absolută a eșantioanelor de ieșire $|y(n)|$ și sunt stocate într-o linie de întârziere (Delay Line) pentru ultimele M eșantioane.
- Se calculează media S a acestor valori absolute după formula: $S = (1/M) \cdot \sum |y(n-k)|$, pentru $k = 0$ până la $M-1$

- **Eroarea:** Media S este comparată cu o valoare de referință fixă REF , diferența dintre ele indicând dacă semnalul este prea puternic sau prea slab.
- **Ajustarea câștigului:** Pe baza acestei erori, câștigul se actualizează conform relației: $g(n) = g(n-1) + \mu \cdot [REF - S]$ unde μ reprezintă viteza de adaptare — o valoare mică înseamnă adaptare lentă și stabilă, iar una mare înseamnă reacție rapidă dar potențial instabilă.

Parametrii funcției sunt:

- **REF** — nivelul țintă al semnalului de ieșire
- **M** — numărul de eșantioane folosite pentru calculul mediei
- **μ** — viteza de adaptare a câștigului

- **Adaptive Line Enhancer (ALE)**



Adaptive Line Enhancer (ALE) - este un algoritm de procesare adaptivă care are capacitatea de a separa componentele periodice (tonuri, sinusoide) de zgomotul aleator dintr-un semnal mixt, îmbunătățind astfel calitatea semnalului util.

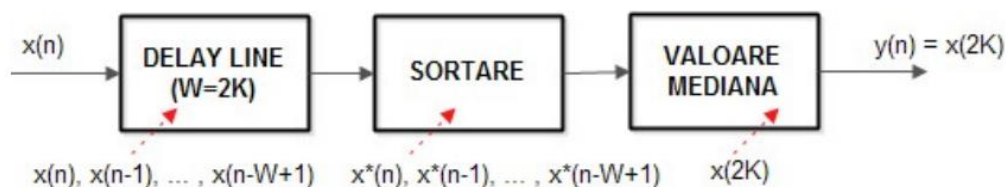
Aceasta este o funcție avansată folosită pentru a detecta și extrage un semnal sinusoidal (sau periodic) dintr-un zgomot de fundal puternic.

Cum funcționează:

- Semnalul de intrare $x(n)$ este combinat cu o versiune întârziată $z(n)$, înmulțită cu coeficientul a , formând semnalul compus $xz(n) = x(n) + a \cdot z(n)$, care simulează condițiile unui mediu real zgomotos.
- Semnalul $xz(n)$ trece printr-un element de întârziere z^{-D} (decorrelator), care are rolul de a rupe corelația dintre componentele de zgomot aleatoriu, păstrând în același timp legătura pentru semnalele periodice.
- **Filtrul Adaptiv:** Un set de coeficienți $h(k)$ este ajustat continuu pentru a reconstrui cât mai fidel componenta periodică a semnalului. Ieșirea filtrului se calculează astfel: $y(n) = \sum h(k) \cdot xz(n-k-D)$, pentru $k = 0, 1, \dots, N-1$
- **Eroarea $e(n)$:** Reprezintă diferența dintre semnalul compus $xz(n)$ și ieșirea filtrată $y(n)$. Această eroare este folosită pentru a actualiza coeficienții filtrului la fiecare pas, conform relației: $h(k) = (1 - \mu \cdot \lambda) \cdot h(k) + \mu \cdot xz(n-k-D) \cdot e(n)$ Practic, filtrul învață în timp să recunoască și să reproducă componenta periodică, eliminând zgomotul.

Parametrii funcției sunt:

- $k(N)$ — lungimea filtrului adaptiv (numărul de coeficienți)
 - a — coeficientul de combinare a semnalului întârziat
 - μ — viteza de adaptare a coeficienților
 - λ — factorul de uitare (controlează stabilitatea filtrului)
- **Median Filter (MF)**



Median Filter (MF) - este un filtru neliniar digital de procesare a semnalului care elimină zgomotul impulsiv (eșantioane eronate cu valori foarte mari sau foarte mici) dintr-un semnal, păstrând în același timp forma generală a acestuia fără a-l deforma.

Cum funcționează:

- **Fereastra de eșantionare:** Sistemul reține în permanență ultimele $W = 2K$ eșantioane ale semnalului de intrare într-o linie de întârziere (Delay Line),

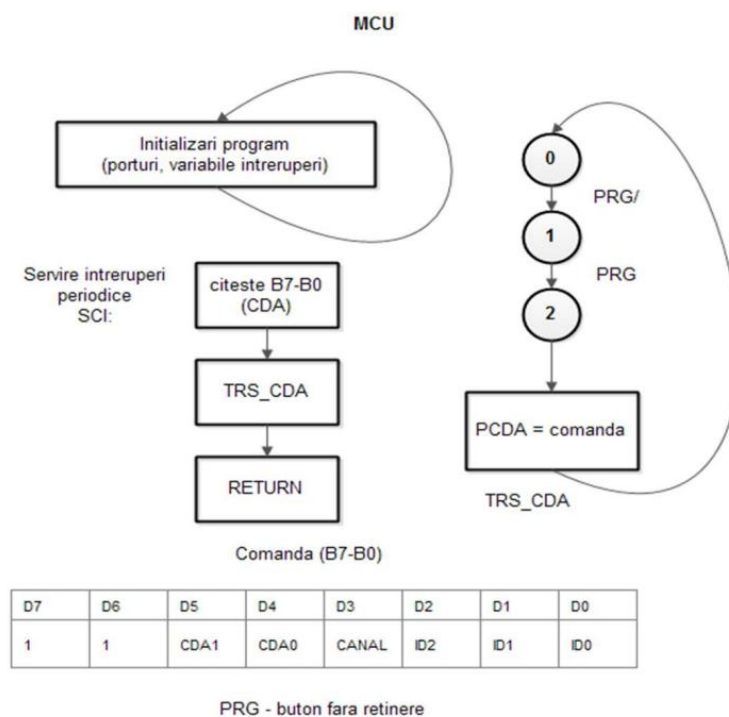
formând fereastra de analiză ($x(n)$, $x(n-1)$, ..., $x(n-W+1)$), unde dimensiunea ferestrei este controlată de parametrul K .

- **Sortarea:** Toate valorile din această fereastră sunt aranjate în ordine crescătoare, obținându-se șirul sortat $x^*(n)$, $x^*(n-1)$, ..., $x^*(n-W+1)$.
- **Selecția Mediane:** Spre deosebire de un filtru de medie aritmetică (care sumează și împarte valorile), filtrul median selectează pur și simplu **valoarea din mijlocul** listei sortate — $x(2K)$ — ca eșantion de ieșire $y(n)$. Aceasta este valoarea mediană a ferestrei.
- **Rezultat:** Valorile extreme cauzate de zgomotul impulsiv (pârâituri, clicuri) sunt ignorate complet, deoarece vor fi întotdeauna la capetele listei sortate, rezultând un semnal de ieșire mult mai curat.

Parametrul funcției este:

- **K** — jumătate din dimensiunea ferestrei de sortare ($W=2K$ eșantioane)

3. Logica programului ARM



Microcontrolerul ARM joacă rolul de Master în cadrul sistemului. Acesta nu intervine în procesarea semnalului audio, ocupându-se doar de preluarea comenzilor de la utilizator și transmiterea lor către DSP.

La pornirea sistemului sunt realizate inițializările necesare — setarea porturilor și pregătirea variabilelor de întreruperi — după care programul intră într-o buclă de așteptare.

1. Inițializare și Așteptare

- **Configurare porturi:** La start, ARM-ul setează butoanele B7-B0 și PRG drept intrări, iar LED-urile și portul PCDA drept ieșiri.
- **Întreruperi periodice (SCI):** Rulează în fundal și are misiunea de a citi periodic starea butoanelor B7-B0 și de a pregăti octetul de comandă.

2. Mașina de Stări pentru Transmisie (Ciclul 0-1-2)

Transmiterea comenzii către DSP este coordonată de un automat cu trei stări:

- **Starea 0:** Stare de repaus. Sistemul verifică continuu butonul PRG — atât timp cât acesta nu e apăsat (PRG/ = nivel ridicat), nu se întâmplă nimic. O apăsare detectată mută execuția în starea 1.
- **Starea 1:** Se confirmă apăsarea și se preiau valorile de pe butoanele B7-B0 prin portul CDA, din care se assemblează comanda ce urmează a fi transmisă.
- **Starea 2:** Comanda asamblată este pusă pe portul PCDA și se apelează TRS_CDA pentru a finaliza transferul către DSP, după care automatul se întoarce în starea 0.

Comanda

D7	D6	D5	D4	D3	D2	D1	D0
1	1	CDA1	CDA0	CANAL	ID2	ID1	ID0

1. Biții de Sincronizare (D7, D6)

- Sunt întotdeauna 1 și servesc drept marcaj de validare. DSP-ul îi verifică primul — dacă nu sunt ambii 1, comanda e respinsă și sistemul poate intra în bypass.

2. Codul Parametrului (D5, D4 - CDA)

- Precizează care parametru al funcției curente va fi actualizat. DSP-ul folosește acești biți pentru a ști dacă trebuie să citească o valoare nouă de la SW7-SW0 (ex: μ la ALE, K la MF).

3. Canalul Audio (D3)

- Indică pe ce canal se aplică procesarea — 0 pentru stânga, 1 pentru dreapta — permițând tratarea independentă a celor două canale.

4. Funcția Activă (D2, D1, D0 - ID)

- ID = 0 (000): Se activează funcția AGC.
- ID = 1 (001): Se activează funcția ALE.
- ID = 2 (010): Se activează funcția MF.

4. Logica programului DSP

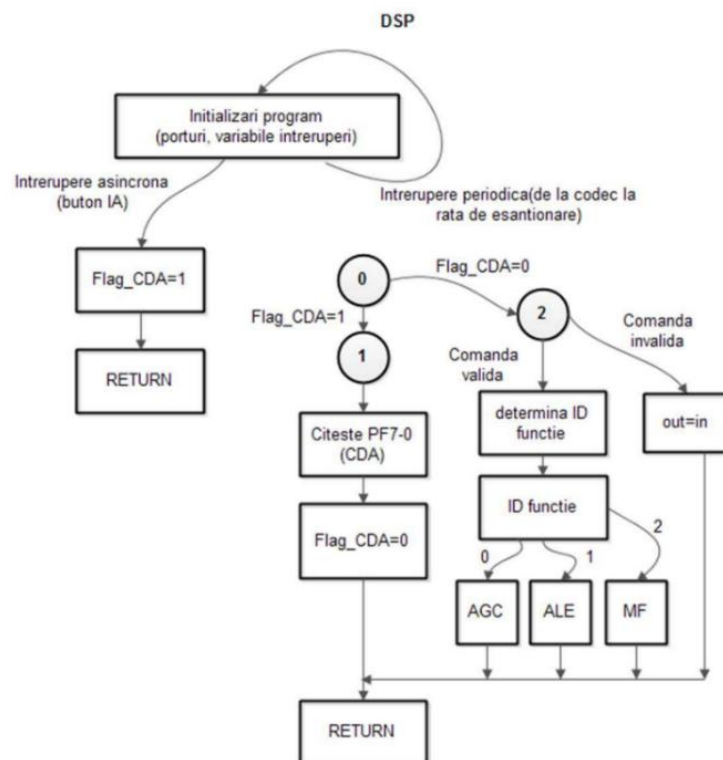


Figura 4. Funcțiile implementate pe DSP

DSP-ul are rolul de a procesa semnalul primit de la CODEC și de a aplica funcția selectată de ARM. Programul DSP este condus de două surse de întreruperi, vizibile în organigrama din Figura 4.

1. Faza de Inițializare

La pornirea sistemului se configurează porturile (PF7-0 ca intrare pentru date și pini de control), variabilele globale și vectorii de întreruperi. După finalizarea inițializării, DSP-ul intră într-o stare de așteptare guvernată de două tipuri de întreruperi: **asincronă** (de la ARM) și **periodică** (de la Codec).

2. Întreruperea Asincronă (buton IA)

Această întrerupere funcționează independent de fluxul audio și asigură preluarea rapidă a comenzilor de la ARM.

- **Eveniment:** ARM-ul activează pinul IA al DSP-ului semnalizând o comandă nouă.
- **Acțiune:** DSP-ul întrerupe execuția curentă, setează $\text{Flag_CDA} = 1$ și revine imediat prin RETURN. Flag-ul are rolul de memorie tampon, anunțând că pe portul PF7-0 așteaptă o comandă nouă.

3. Întreruperea Periodică (de la CODEC, la rata de eșantionare)

Aceasta este întreruperea principală, care rulează la rata de eșantionare a Codec-ului. Fiecare eșantion $x(n)$ trece prin următoarea mașină de stări:

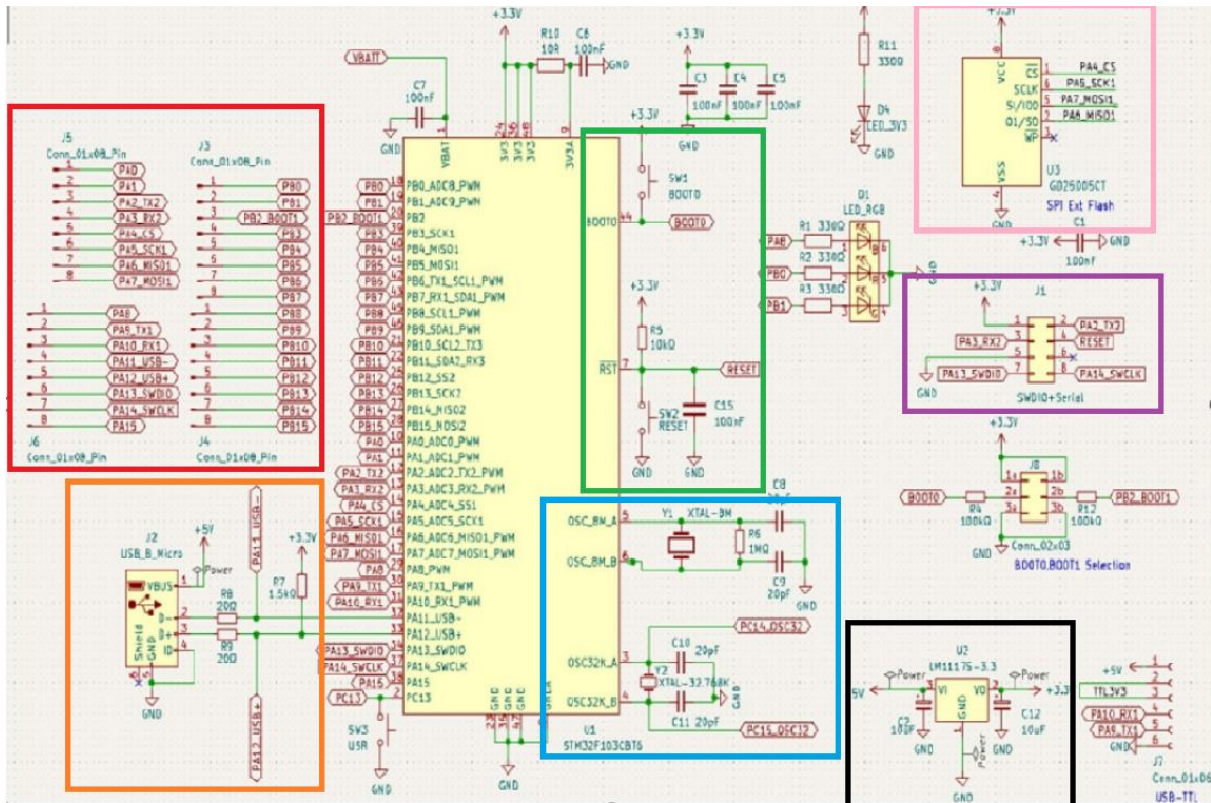
- **Starea 0 — Verificarea Flag-ului:**
 - Dacă $\text{Flag_CDA} = 1 \rightarrow$ există o comandă nouă, se trece în Starea 1.
 - Dacă $\text{Flag_CDA} = 0 \rightarrow$ nu a sosit nicio comandă, se sare direct în Starea 2 cu setările existente.
- **Starea 1 — Preluarea Comenzii:**
 - DSP-ul citește byte-ul de pe portul PF7-0 (CDA).
 - Flag_CDA este resetat la 0 pentru a marca că informația a fost preluată.
 - Se trece automat în Starea 2.
- **Starea 2 — Execuția Algoritmului:**
 - DSP-ul analizează biții D2-D0 pentru a identifica funcția dorită.
 - Dacă comanda e validă ($D7=D6=1$), se execută funcția corespunzătoare: AGC, ALE sau MF.
 - Dacă comanda e invalidă $\rightarrow$ se execută bypass ($\text{out} = \text{in}$), eșantionul de intrare $x(n)$ fiind trimis direct la ieșire fără modificări.

4. Finalizarea Ciclului (RETURN)

După procesarea eșantionului prin una din cele trei funcții sau prin bypass, se execută RETURN. DSP-ul intră în repaus până la următorul semnal de tact de la Codec, reluând întreg procesul din Starea 0.

5. Scheme electrice și configurarea pinilor - ARM

Schema electrică ARM



LEDs: Legate la pinii PB0-PB7.

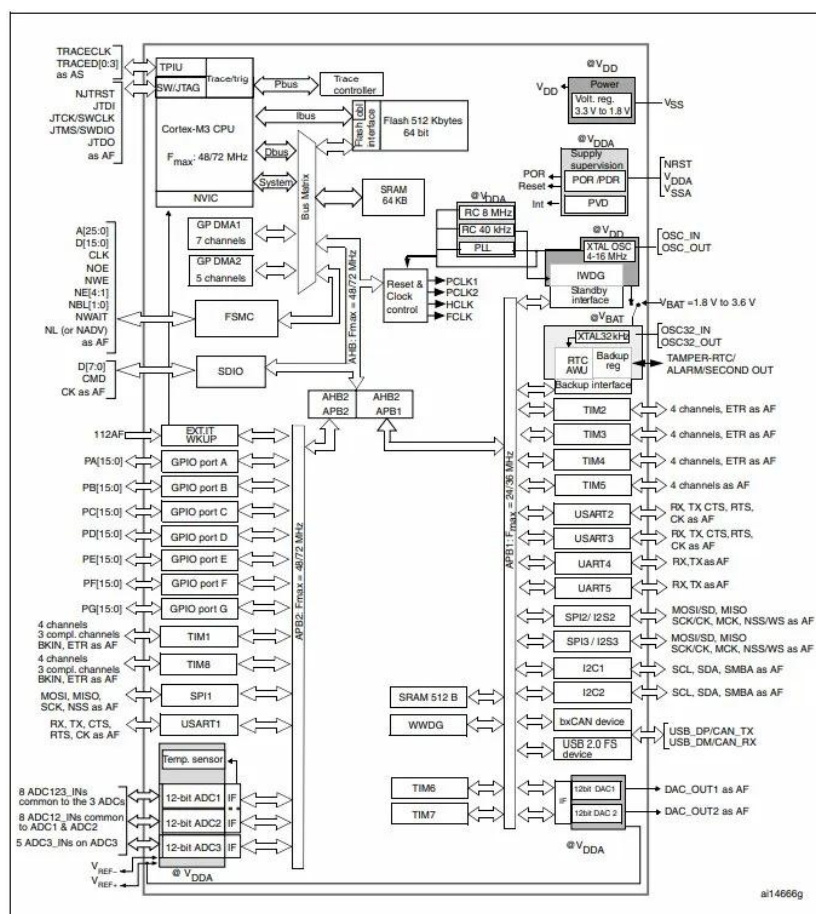
Buttons: Legate la pinii PA1-PA8.

CDA: Legat la pinii PB8-PB15 (interfața către DSP).

- **Blocul roșu:** Conține conectorii I/O și Serial. Conectorii J3, J4, J5 și J6 sunt GPIO Headers, adică pini de expansiune ai microcontrollerului, utilizabili pentru conectarea de senzori sau pentru diverse tipuri de comunicație.
- **Blocul portocaliu:** Reprezintă interfața USB Micro-B (J2), folosită atât pentru programarea microcontrollerului, cât și pentru alimentarea sa. Liniile de date D+ și D- trec prin rezistențele R7 și R8 pentru a asigura compatibilitatea cu protocolul USB.

- **Blocul verde:** Găzduiește selecția sursei de boot (BOOT0), care determină dacă microcontrollerul pornește din Flash, RAM sau UART. J8 este utilizat pentru alegerea modului de boot. De asemenea, SW2 este un buton de reset manual, iar R5 este o rezistență de Pull-Up, esențială pentru menținerea unui nivel logic stabil.
- **Blocul albastru deschis:** Include două oscilatoare. Y1 este oscilatorul principal cu frecvența de 8 MHz, iar Y2 este un oscilator RTC (Real-Time Clock) de 32.768 kHz. Componentele auxiliare, cum ar fi condensatoarele și rezistențele, contribuie la stabilitatea funcționării acestora.
- **Blocul roz:** Conține o memorie SPI Flash externă, de tip non-volatil, ceea ce înseamnă că datele stocate nu se pierd la întreruperea alimentării. Aceasta include pini precum WP (Write Protect) și Hold, care oferă protecție suplimentară și control asupra accesului la memorie. Protocolul SPI (Serial Peripheral Interface) este utilizat pentru comunicarea sincronă între microcontroller și memorie, folosind liniile MOSI (Master Out Slave In) și MISO (Master In Slave Out).
- **Blocul mov:** Reprezintă un conector J1 cu 8 pini, ce combină interfața de debug și programare Serial Wire Debug (SWD) cu interfața UART (serială). PA2_TX2 este linia de transmisie UART de la STM32 către PC, iar PA3_RX2 este linia de recepție de la PC către STM32. Pini PA13_SWDIO și PA14_SWCLK sunt folosiți pentru depanare și programare, esențiali pentru încărcarea firmware-ului prin ST-Link.
- **Blocul negru:** Este un regulator de tensiune care convertește 5V în 3.3V, necesari pentru alimentarea microcontrollerului. Condensatoarele C12 și C13 au rolul de a stabili tensiunea de ieșire și de a reduce fluctuațiile.

Schema bloc STM32F103

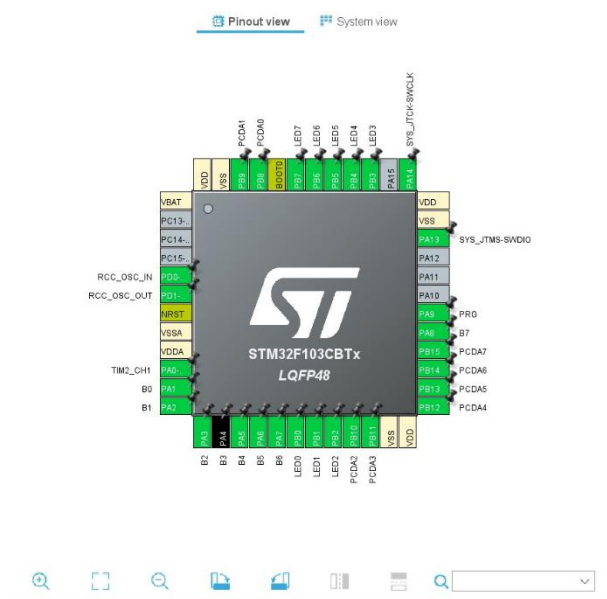


Schema bloc din figura de mai sus prezintă arhitectura internă a microcontrolerului STM32F103. Principalele componente sunt:

- **Cortex-M3 CPU** – nucleul procesorului, cu frecvență maximă de 48/72 MHz, conectat la memorie prin bus-urile Ibus și Dbus
- **Memorie** – Flash 512 KB pentru stocarea programului și SRAM 64 KB pentru date
- **NVIC** – controlerul de întreruperi, gestionează toate întreruperile sistemului
- **Bus Matrix** – interconectează CPU cu perifericele prin magistralele AHB, APB1 și APB2
- **GPIO** – porturi de intrare/ieșire (PA–PG), folosite pentru butoane, LED-uri și comunicația cu DSP-ul
- **Timere** – TIM1–TIM8, dintre care noi folosim TIM2 pentru generarea întreruperii periodice
- **Alimentare** – tensiune de lucru 3.3V–1.8V, cu supervizare PVD și oscilator intern RC 8 MHz / extern XTAL 4–16 MHz

Schema de configurare a pinilor (STM32CubeIDE – Pinout View)

În figura de mai jos este prezentată configurarea pinilor microcontrolerului STM32F103CBTx realizată în STM32CubeIDE.



Pinii au fost configurați astfel:

Intrări (GPIOA): PA1–PA8 sunt configurați ca intrări digitale (Input mode, fără pull-up/pull-down) și corespund butoanelor B0–B7. PA9 este de asemenea intrare și corespunde butonului PRG.

Ieșiri (GPIOB): PB0–PB7 sunt configurați ca ieșiri push-pull (Output Push Pull, nivel inițial Low) și controlează LED-urile LED0–LED7 pentru afișarea stării interne a sistemului ARM. PB8–PB15 sunt tot ieșiri push-pull și formează portul de comandă PCDA0–PCDA7, prin care se transmite comanda către DSP.

Oscilator (GPIOD): PD0 și PD1 sunt rezervați pentru oscilatorul extern (RCC_OSC_IN și RCC_OSC_OUT), folosit ca sursă de ceas pentru PLL.

Configuration							
Group By Peripherals							
GPIO RCC SYS TIM							
Search Signals							
Search (Ctrl+F)							
☐ Show only Modified Pins							
Pin Name	Signal on Pin	GPIO output level	GPIO mode	GPIO Pull-up/Pull-down	Maximum output...	User Label	Modified
PA1	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	B0	☒
PA2	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	B1	☒
PA3	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	B2	☒
PA4	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	B3	☒
PA5	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	B4	☒
PA6	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	B5	☒
PA7	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	B6	☒
PA8	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	B7	☒
PA9	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	PRG	☒

PB0	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED0	☒
PB1	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED1	☒
PB2	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED2	☒
PB3	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED3	☒
PB4	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED4	☒
PB5	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED5	☒
PB6	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED6	☒
PB7	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED7	☒
PB8	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA0	☒
PB9	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA1	☒
PB10	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA2	☒

PB5	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED5	☒
PB6	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED6	☒
PB7	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED7	☒
PB8	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA0	☒
PB9	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA1	☒
PB10	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA2	☒
PB11	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA3	☒
PB12	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA4	☒
PB13	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA5	☒
PB14	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA6	☒
PB15	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA7	☒

Configuration							
Group By Peripherals							
☒ GPIO ☒ RCC ☒ SYS ☒ TIM							
Search Signals							
Search (Ctrl+F) ☐ Show only Modified Pins							
Pin Name	Signal on Pin	GPIO output level	GPIO mode	GPIO Pull-up/Pull-	Maximum output s.	User Label	Modified
PD0-OSC_IN	RCC_OSC_IN	n/a	n/a	n/a	n/a		☐
PD1-OSC_OUT	RCC_OSC_OUT	n/a	n/a	n/a	n/a		☐

6. Cod – STM32

/* USER CODE BEGIN Header */

/**

* @file : main.c

* @brief : Main program body

* @attention

*

* Copyright (c) 2026 STMicroelectronics.

* All rights reserved.

*

* This software is licensed under terms that can be found in the LICENSE file

* in the root directory of this software component.

* If no LICENSE file comes with this software, it is provided AS-IS.

*

*/

/* USER CODE END Header */

/* Includes -----*/

#include "main.h"

/* Private includes -----*/

/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----*/

/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define -----*/

/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -----*/

/* USER CODE BEGIN PM */


```
/* USER CODE END PM */
```

```
/* Private variables -----*/
```

```
TIM_HandleTypeDef htim2;
```

```
/* USER CODE BEGIN PV */
```

```
uint8_t comanda_dsp = 0;
```

```
uint8_t stare_trs = 0;
```

```
/* USER CODE END PV */
```

```
/* Private function prototypes -----*/
```

```
void SystemClock_Config(void);
```

```
static void MX_GPIO_Init(void);
```

```
static void MX_TIM2_Init(void);
```

```
/* USER CODE BEGIN PFP */
```

```
/* USER CODE END PFP */
```

```
/* Private user code -----*/
```

```
/* USER CODE BEGIN 0 */
```

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
```

```
{
```

```
    // Aceasta functie este apelata automat la fiecare perioada a timerului
```

```
    // Verificam daca sursa intreruperii este TIM2 (pot exista mai multi timeri activi)
```

```
    if (htim->Instance == TIM2)
```

```
    {
```

```
        // Preia nivelul logic actual al pinului PRG de pe portul GPIOA
```

```
GPIO_PinState prg = HAL_GPIO_ReadPin(GPIOA, PRG_Pin);
```

```
// Automatul de stari pentru gestionarea transmisiei comenzii
```

```
switch (stare_trs)
```

```
{
```

```
case 0:
```

```
    // STAREA 0: Monitorizare eliberare buton PRG
```

```
    // Ne asiguram ca butonul a fost complet eliberat dupa
```

```
    // o apasare anterioara, inainte de a permite o noua comanda
```

```
    if (prg == GPIO_PIN_RESET)
```

```
    {
```

```
        stare_trs = 1; // Butonul e liber -> sistem pregatit pentru o noua apasare
```

```
    }
```

```
    // Cat timp PRG ramane apasat, starea nu se schimba
```

```
    break;
```

```
case 1:
```

```
    // STAREA 1: Detectie apasare buton PRG si construire comanda
```

```
    // Sistemul asteapta ca utilizatorul sa apese PRG
```

```
    // pentru a captura valorile butoanelor B0-B7
```

```
    if (prg == GPIO_PIN_SET)
```

```
    {
```

```
        // Citeste toti pinii portului A dintr-o singura operatie
```

```
        uint16_t stare_port_A = GPIOA->IDR;
```

```
        // Extrage bitii B0-B5 de pe PA1-PA6:
```

```
        // >> 1 decaleaza pentru a elimina PA0, & 0x3F pastreaza doar 6 biti
```

```
        uint8_t date_butoane = (uint8_t)((stare_port_A >> 1) & 0x3F);
```

```

// Asambleaza octetul de comanda:
// 0xC0 seteaza D7=1 si D6=1 (biti de validare)
// OR cu date_butoane adauga bitii D5-D0 (parametru + canal + functie)
comanda_dsp = 0xC0 | date_butoane;

stare_trs = 2; // Comanda construita -> gata de transmisie
}

// Daca PRG inca nu e apasat, ramanem in asteptare
break;

case 2:
// STAREA 2: Plasarea comenzii pe portul de iesire
// Scriem simultan in ambele jumatati ale registrului ODR al portului B:
// - Jumatatea superioara (PB8-PB15) = PCDA7-PCDA0 -> date catre DSP
// - Jumatatea inferioara (PB0-PB7) = LED7-LED0 -> afisare vizuala a comenzii
GPIOB->ODR = ((uint16_t)comanda_dsp << 8) | (uint16_t)comanda_dsp;

stare_trs = 0; // Transmisie finalizata -> revenire in starea de monitorizare
break;

default:
stare_trs = 0; // Stare necunoscuta -> resetare automata la starea initiala
break;
}
}
}

/* USER CODE END 0 */

/**

```

```

* @brief The application entry point.
* @retval int
*/
int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_TIM2_Init();

    /* USER CODE BEGIN 2 */

    HAL_TIM_Base_Start_IT(&htim2); // Pornește Timer 2 în regim de întreruperi

```

```
/* USER CODE END 2 */
```

```
/* Infinite loop */
```

```
/* USER CODE BEGIN WHILE */
```

```
while (1)
```

```
{
```

```
/* USER CODE END WHILE */
```

```
/* USER CODE BEGIN 3 */
```

```
}
```

```
/* USER CODE END 3 */
```

```
}
```

```
/**
```

```
 * @brief System Clock Configuration
```

```
 * @retval None
```

```
 */
```

```
void SystemClock_Config(void)
```

```
{
```

```
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
```

```
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
```

```
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
```

```
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
```

```
  RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
```

```
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
```

```
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
```

```
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
```

```

RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9; // 8MHz * 9 = 72MHz
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK) { Error_Handler(); }

RCC_ClkInitStruct.ClockType                                     =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                        |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) !=
HAL_OK) { Error_Handler(); }
}

/**
 * @brief TIM2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM2_Init(void)
{

/* USER CODE BEGIN TIM2_Init 0 */
/* USER CODE END TIM2_Init 0 */
TIM_ClockConfigTypeDef sClockSourceConfig = {0};
TIM_MasterConfigTypeDef sMasterConfig = {0};
TIM_OC_InitTypeDef sConfigOC = {0};
/* USER CODE BEGIN TIM2_Init 1 */

```

```

/* USER CODE END TIM2_Init 1 */

htim2.Instance = TIM2;
htim2.Init.Prescaler = 7199;
htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
htim2.Init.Period = 100;
htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
{
    Error_Handler();
}
sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
{
    Error_Handler();
}
if (HAL_TIM_OC_Init(&htim2) != HAL_OK)
{
    Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) !=
HAL_OK)
{
    Error_Handler();
}

```

```

sConfigOC.OCMode = TIM_OCMode_TIMING;
sConfigOC.Pulse = 0;
sConfigOC.OCpolarity = TIM_OCPolarity_HIGH;
sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
if (HAL_TIM_OC_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_1) !=
HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN TIM2_Init 2 */

/* USER CODE END TIM2_Init 2 */

HAL_TIM_MspPostInit(&htim2);
}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
/* USER CODE BEGIN MX_GPIO_Init_1 */
/* USER CODE END MX_GPIO_Init_1 */

/* GPIO Ports Clock Enable */
__HAL_RCC_GPIOA_CLK_ENABLE();

```



```

__HAL_RCC_GPIOB_CLK_ENABLE();

/*Configure GPIO pin Output Level */

HAL_GPIO_WritePin(GPIOB, LED0_Pin|LED1_Pin|LED2_Pin|PCDA2_Pin
                  |PCDA3_Pin|PCDA4_Pin|PCDA5_Pin|PCDA6_Pin
                  |PCDA7_Pin|LED3_Pin|LED4_Pin|LED5_Pin
                  |LED6_Pin|LED7_Pin|PCDA0_Pin|PCDA1_Pin,
GPIO_PIN_RESET);

GPIO_InitStruct.Pin = B0_Pin|B1_Pin|B2_Pin|B3_Pin
                    |B4_Pin|B5_Pin|B6_Pin|B7_Pin
                    |PRG_Pin;

GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);


GPIO_InitStruct.Pin = LED0_Pin|LED1_Pin|LED2_Pin|PCDA2_Pin
                    |PCDA3_Pin|PCDA4_Pin|PCDA5_Pin|PCDA6_Pin
                    |PCDA7_Pin|LED3_Pin|LED4_Pin|LED5_Pin
                    |LED6_Pin|LED7_Pin|PCDA0_Pin|PCDA1_Pin;

GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

/* USER CODE BEGIN MX_GPIO_Init_2 */
/* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

```

```
/* USER CODE END 4 */
```

```
/**
```

```
 * @brief This function is executed in case of error occurrence.
```

```
 * @retval None
```

```
 */
```

```
void Error_Handler(void)
```

```
{
```

```
 /* USER CODE BEGIN Error_Handler_Debug */
```

```
 /* User can add his own implementation to report the HAL error return state */
```

```
 __disable_irq();
```

```
 while (1)
```

```
 {
```

```
 }
```

```
 /* USER CODE END Error_Handler_Debug */
```

```
}
```

```
#ifdef USE_FULL_ASSERT
```

```
/**
```

```
 * @brief Reports the name of the source file and the source line number
```

```
 *      where the assert_param error has occurred.
```

```
 * @param file: pointer to the source file name
```

```
 * @param line: assert_param error line source number
```

```
 * @retval None
```

```
 */
```

```
void assert_failed(uint8_t *file, uint32_t line)
```

```
{
```

```
 /* USER CODE BEGIN 6 */
```

```
 /* User can add his own implementation to report the file name and line number,
```

```

ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */

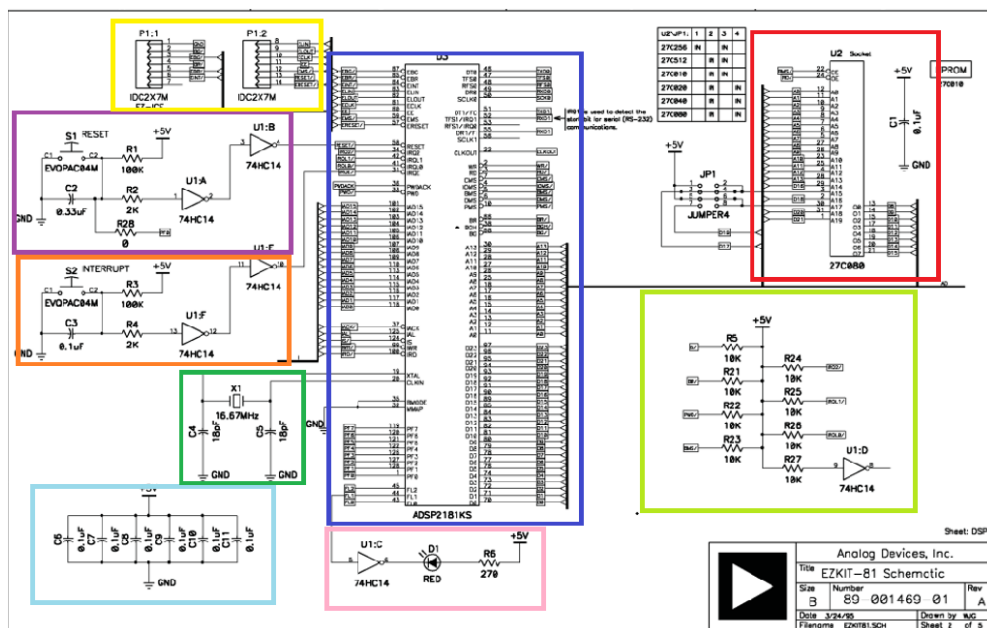
/* USER CODE END 6 */

}

#endif /* USE_FULL_ASSERT */

```

7. Scheme electrică - DSP



Blocul evidențiat cu **roșu** conține o memorie de tip EPROM (Erasable Programmable Read-Only Memory), utilizată pentru stocarea permanentă a firmware-ului care rulează pe DSP. Această memorie este activată prin semnalul CE (Chip Enable), în timp ce semnalul OE (Output Enable) controlează activarea liniilor de ieșire pentru citirea datelor. Magistrala de adresă este formată din 20 de linii (A0–A19), permițând accesul la un spațiu de 1 MB (2^{20} locații de memorie), iar datele sunt transmise pe liniile O0–O7. Dimensiunea activă a memoriei poate fi configurată prin jumperul JP1.

Blocul central, evidențiat cu **albastru închis**, reprezintă nucleul procesorului digital de semnal. Acesta are o arhitectură de tip CISC (Complex Instruction Set Computer), optimizată pentru execuția eficientă a operațiilor aritmetice și logice complexe, tipice aplicațiilor de procesare a semnalelor. Magistrala de date/adrese este multiplexată pe liniile IAD0–IAD15, în timp ce liniile A0–A13 sunt utilizate pentru adresarea directă a memoriei. Controlul operațiilor de memorie este realizat prin liniile WR și RD, iar întreruperile externe sunt gestionate prin pinii IRQ2, IRQ1, IRQ0 și IRDE. Tot în

această zonă (blocul **roz**) este prezent un LED de stare, conectat printr-un circuit limitator de curent format din rezistorul R6 și dioda D1, utilizat pentru semnalizarea vizuală a anumitor evenimente sau stări logice.

În partea stângă, blocul **verde închis** conține oscilatorul principal al sistemului, notat cu X1, care funcționează la o frecvență de 16.67 MHz. Acesta generează semnalul de tact necesar sincronizării proceselor interne ale DSP-ului, prin pinul CLKIN. Condensatoarele C4 și C5 sunt conectate la acest oscilator pentru a-i asigura stabilitatea și funcționarea corectă.

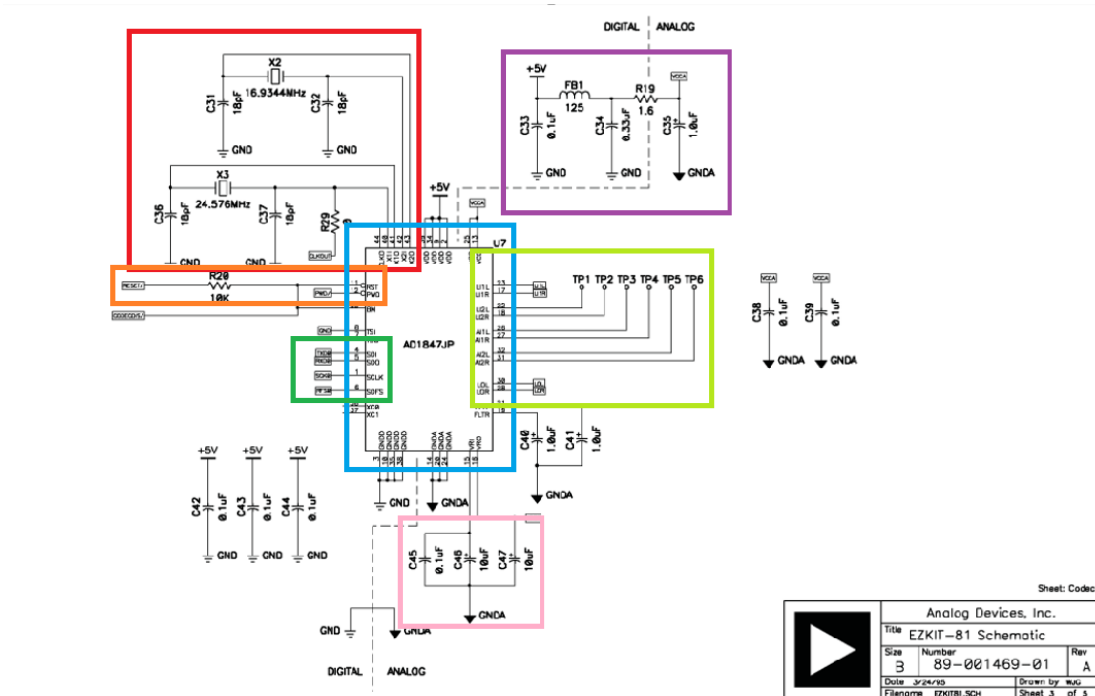
Blocul **mov** integrează un circuit de reset hardware, care permite inițializarea întregului sistem prin apăsarea butonului S1. Semnalul de reset este condiționat printr-un circuit bazat pe inversorul logic 74HC14, cu trigger Schmitt, care elimină zgomotele și asigură un semnal clar. Rezistorul R1 menține linia de reset într-o stare logică înaltă în mod normal, iar condensatorul C1 se încarcă lent pentru a introduce o întârziere controlată. În completare, condensatorul C2 contribuie la generarea unui impuls de reset coerent și stabil.

Blocul **portocaliu** este destinat generării unei întreruperi externe, activată prin apăsarea butonului S2. Prin această metodă, utilizatorul poate simula sau introduce evenimente externe care necesită tratare prioritară de către DSP, permițând declanșarea unei rutine de întrerupere.

Blocul evidențiat cu **galben** reprezintă interfața EZ-ICE, dedicată procesului de depanare. Aceasta permite conectarea unui debugger sau emulator extern pentru analiza în timp real a execuției codului, facilitând inspecția registrelor, monitorizarea stărilor interne și încărcarea firmware-ului direct în memoria DSP-ului, fără a întrerupe funcționarea generală a sistemului.

În partea dreaptă, blocul **verde deschis** include un circuit de tip pull-up, utilizat pentru a menține anumite linii de control (precum SR, PWDR sau IRQ) într-o stare logică înaltă atunci când acestea nu sunt active. Prin utilizarea rezistențelor de pull-up, se previne apariția stărilor necontrolate sau a fluctuațiilor pe aceste linii. Totodată, inversorul 74HC14 contribuie la stabilizarea logică a semnalelor și la reducerea zgomotului electric de pe magistrală.

În final, blocul **albastru deschis** este format dintr-un ansamblu de condensatori de decuplare, care contribuie la stabilizarea sursei de alimentare prin eliminarea zgomotului de înaltă frecvență și reducerea interferențelor electromagnetice. Aceștia sunt esențiali în menținerea unei alimentări curate și stabile pentru componentele digitale, mai ales în circuitele cu comutații rapide.



În zona marcată cu **roșu** sunt evidențiate două oscilatoare de tip cristal, notate X2 și X3, care generează semnalul de ceas necesar funcționării corecte a codec-ului AD1847JP. Aceste oscilatoare furnizează semnalul de tact necesar sincronizării interne a conversiilor și a fluxurilor de date digitale. Condensatoarele C31, C32, C36 și C37 sunt conectate în proximitatea cristalelor și joacă un rol esențial în stabilizarea frecvenței de oscilație, reducând variațiile nedorite și asigurând o funcționare predictibilă a circuitelor de clock.

În blocul **portocaliu** este prezent un buton de reset hardware, utilizat pentru inițializarea codec-ului audio. Acesta este menținut în stare logică înaltă (HIGH) prin intermediul rezistorului R20, care asigură o stare stabilă pe linia de reset în lipsa acționării butonului. La apăsare, semnalul este tras la nivel jos (LOW), declanșând astfel o resetare a componentelor conectate.

Zona marcată cu **albastru** conține codec-ul audio propriu-zis, AD1847JP, un circuit integrat care realizează conversia bidirecțională între semnale digitale și semnale analogice. Acesta funcționează ca interfață între procesorul de semnal digital (DSP) și lumea analogică, oferind atât conversie digital-analog (DAC), cât și analog-digital (ADC), în mod stereo. Acest tip de componentă este esențială în aplicații de procesare audio.

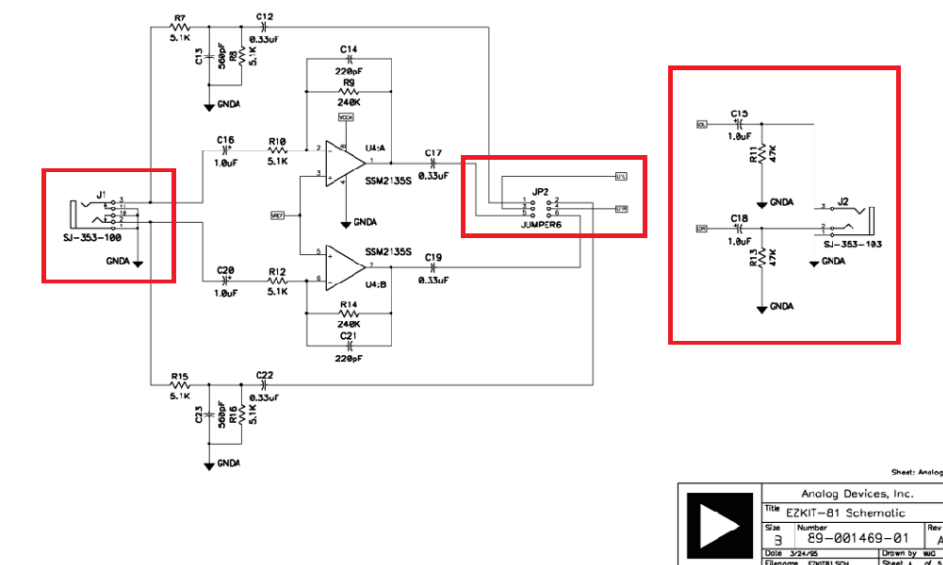
Blocul **verde închis** descrie interfața digitală de comunicație a codec-ului, printr-un set de linii de control seriale. Linia **SDI** (Serial Data Input) este utilizată pentru recepționarea de date digitale dinspre DSP, în timp ce linia **SDO** (Serial Data Output) transmite date digitale către procesor. Semnalul **SCLK** (Serial Clock) sincronizează transferul serial de date, asigurând alinierea corectă a bitilor, iar semnalul **SOFS** (Sync

Frame) marchează începutul unui nou cadru de date audio, funcționând ca referință de sincronizare în protocoalele seriale audio.

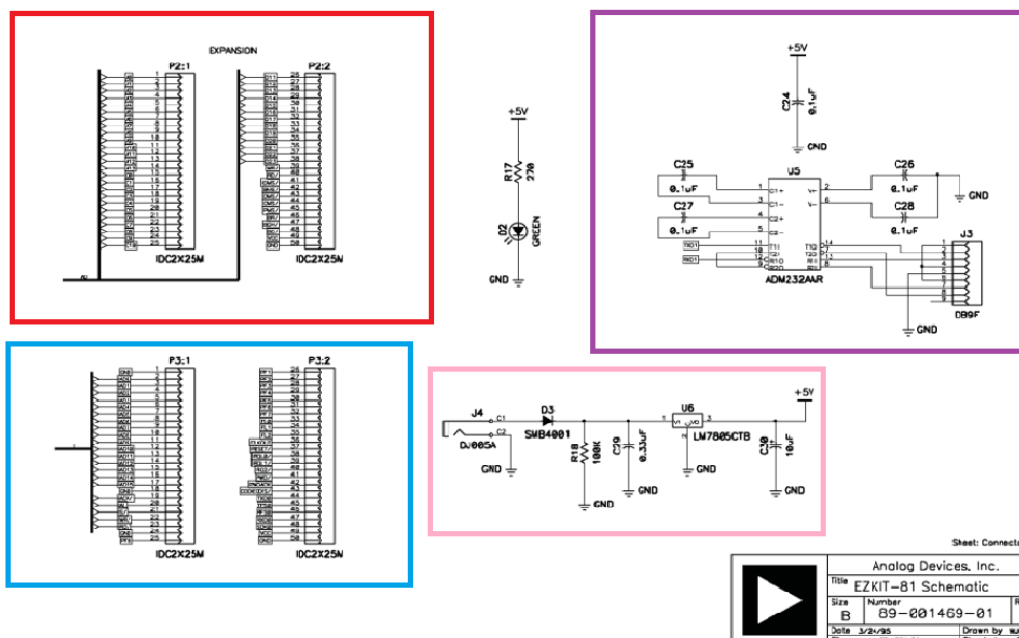
Blocul evidențiat cu **mov** reprezintă un filtru de separare între domeniul de alimentare digital și cel analogic. Acest filtru este esențial pentru prevenirea interferențelor electrice generate de circuitele logice asupra secțiunii audio analogice, mult mai sensibilă la zgomot. Bobina **FB1** (ferrite bead) acționează ca un filtru pasiv de înaltă frecvență, blocând zgomotele care pot trece de pe partea digitală pe partea analogică. În paralel, condensatorul **C35** acționează ca un rezervor local de energie, stabilizând tensiunea de alimentare în zona analogică în fața fluctuațiilor rapide de curent.

Blocul central, evidențiat cu **verde deschis**, include terminalele de intrare și ieșire ale codec-ului AD1847JP, care interfațează direct cu sursele și destinațiile semnalului audio. Intrările **L1R** și **L1L** reprezintă canalul drept și stâng al primei perechi de intrări audio, în timp ce **L2R** și **L2L** oferă o a doua pereche de intrări, utile pentru aplicații stereo avansate sau selecție de sursă. Terminalele **A1L** și **A2R** sunt dedicate conectării microfoanelor sau altor surse analogice auxiliare. Ieșirile **LOL** și **LOR** oferă semnalul audio analogic convertit, direcționat către etaje de amplificare sau ieșiri către utilizator. Pinul **FLTR** este destinat conectării unui filtru analogic extern, care poate fi utilizat pentru îmbunătățirea fidelității semnalului sau pentru eliminarea componentelor de frecvență nedorită.

În final, blocul **roz** evidențiază un circuit dedicat stabilizării tensiunii de referință **VREF**, esențială în procesul de conversie analog-digital și digital-analog. Precizia acestei tensiuni determină acuratețea convertorului, motiv pentru care este necesară o filtrare riguroasă. Condensatorul **C45** atenuează componentele de frecvență înaltă, în timp ce condensatoarele **C46** și **C47** sunt utilizate pentru filtrarea frecvențelor medii și pentru reglarea suplimentară a nivelului de referință, asigurând astfel stabilitate și liniaritate în procesul de conversie.



J1 reprezintă un conector audio Jack, pentru intrarea semnalului audio. J2 este un conector de ieșire pentru semnalul audio. JP2 permite selectare/comutarea între diferite trasee de semnal.



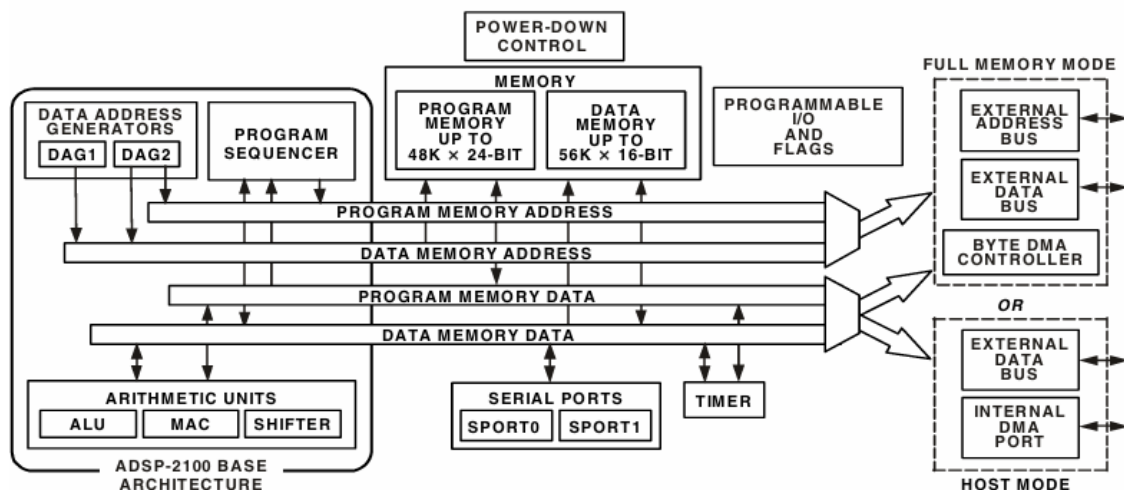
Blocul evidențiat cu **roșu** conține conectori de tip IDC, care asigură accesul la magistrala de adrese și date a microcontrolerului. Liniile A0–A15 sunt utilizate pentru adresarea componentelor de memorie și a perifericelor externe, în timp ce liniile D0–D15 transportă datele bidirecțional, atât pentru operații de citire, cât și de scriere. Aceste magistrale sunt esențiale în arhitecturile Harvard sau Von Neumann, unde accesul paralel la memorie și periferice determină performanța sistemului.

În zona marcată cu **albastru deschis** sunt integrați conectori suplimentari pentru semnale de control și linii de intrare/ieșire, completând accesul oferit de busul principal. Acești conectori includ adrese suplimentare și semnale de control precum RESET, dar și interfețe dedicate pentru comunicații paralele sau conectarea de periferice specifice. Astfel, se oferă flexibilitate în extinderea sistemului cu noi funcționalități hardware.

Blocul marcat cu **mov** include interfața serială a sistemului, destinată comunicării cu un PC sau alt echipament digital. Componenta centrală, U5, este un convertor de niveluri logice care asigură compatibilitatea dintre semnalele de nivel TTL ale microcontrolerului și nivelurile standard RS-232 utilizate pe porturile seriale ale calculatoarelor. Conversia corectă a semnalelor este susținută de o serie de condensatori de 0.1 μF, care sunt necesari pentru funcționarea internă a circuitului de translație a nivelurilor. Conectivitatea fizică este realizată prin intermediul unui conector DB9, conform standardului clasic RS-232.

Blocul evidențiat cu **roz** corespunde circuitului de alimentare al sistemului, fiind responsabil pentru stabilizarea tensiunii de funcționare la 5V. Componenta principală, LM7805, este un regulator de tensiune liniar cu rol de decuplare, care menține o ieșire constantă de 5V, indiferent de variațiile moderate ale tensiunii de intrare. În scopul protejării circuitului, este introdusă o diodă de protecție, D3, care previne deteriorarea sistemului în cazul unei polarizări inverse accidentale a sursei de alimentare. Acest design simplu, dar eficient, este frecvent utilizat în alimentarea stabilă a sistemelor embedded.

Schema bloc ADSP2181



Arhitectura internă se bazează pe structura ADSP-2100 și conține:

- **Unități aritmetice** – ALU, MAC (multiplicator-acumulator) și Shifter, folosite pentru calculele din funcțiile AGC, ALE și MF
- **Program Sequencer** – controlează execuția instrucțiunilor
- **DAG1, DAG2** – generatoare de adrese pentru accesul eficient la bufferele circulare (folosite în codul nostru pentru rx_buf și tx_buf)
- **Memorie Program** – până la $48K \times 24\text{-bit}$ (secțiunile pm_da, interrupts, seg_code din codul nostru)
- **Memorie Date** – până la $56K \times 16\text{-bit}$ (secțiunile buf_var1/2/3, data1 din codul nostru)
- **SPORT0, SPORT1** – porturi seriale; noi folosim SPORT0 pentru comunicația cu CODEC-ul AD1847
- **Programmable I/O and Flags** – pinii PF0–PF7, folosiți pentru citirea comenzii de la ARM

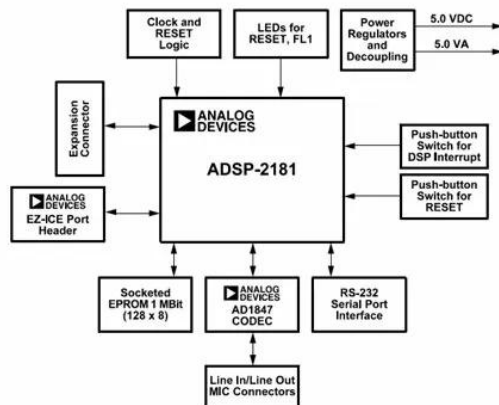
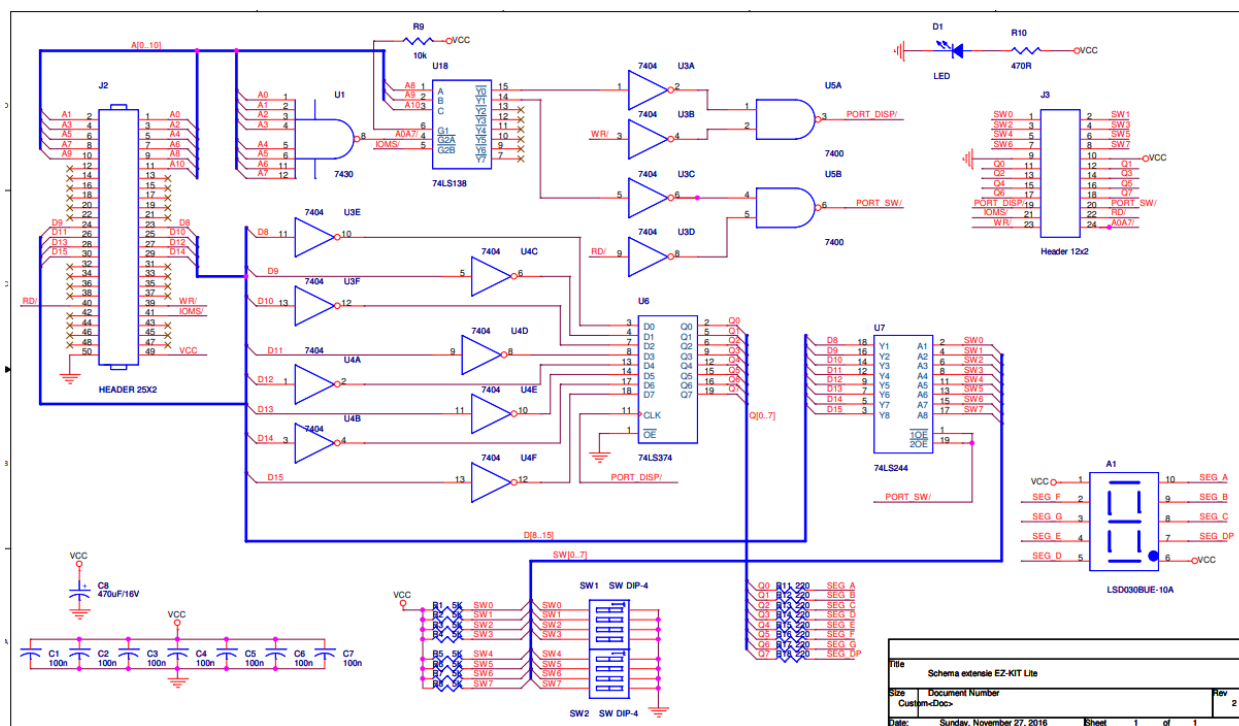


Figure 3-1. ADSP-2181 EZ-KIT Lite System Block Diagram

Extensia I/O DSP



Schema de mai sus reprezintă extensia I/O a plăcii EZ-KIT Lite ADSP-2181. Aceasta asigură interfața dintre DSP și utilizator prin comutatoare și afișor. Principalele componente sunt:

- **HEADER 25x2 (J2)** – conectorul principal prin care extensia se leagă la placa EZ-KIT Lite, aducând bus-ul de date și semnalele de control (RD/, WR/, IOMS/)
- **7430** – poartă NAND cu 8 intrări, folosită pentru decodarea adreselor

- **74LS138 (U18)** – decodor 3-la-8, selectează unul dintre porturile de I/O în funcție de adresă
- **7404** – inversoare logice, folosite pentru adaptarea semnalelor de control (RD/, WR/, PORT_DISP/, PORT_SW/)
- **7400** – porți NAND cu 2 intrări, logică suplimentară de selecție porturi
- **74LS374 (U6)** – registru octet cu latch, reține valoarea scrisă de DSP pentru afișare pe segmente
- **74LS244 (U7)** – buffer pentru citirea stării comutatoarelor SW0–SW7
- **SW1, SW2 (DIP-4)** – comutatoare pentru setarea parametrilor funcțiilor DSP (K pentru MF, etc.)
- **J3 (Header 12x2)** – conector pentru comutatoarele SW0–SW7
- **Afișor cu 7 segmente (A1 – LSD030BUE-10A)** – afișează starea internă a programului DSP (funcția activă)
- **LED (D1) + rezistență 470Ω** – indicator vizual de stare
- **Condensatori de decuplare (C1–C8)** – filtrare alimentare

8. Funcții de procesare

➤ Automatic Gain Control – AGC

```
#include "def2181.h" // registre si periferice ADSP-2181
#define M 4          // numarul de esantioane pentru medie
#define K 2          // factor de scalare (shift dreapta cu 2)

.SECTION/DM data1;

.var/circ rx_buf[3]; // buffer receptie de la codec (intrare audio)
.var/circ tx_buf[3]; // buffer transmisie catre codec (iesire audio)
.var/circ delay[M]; // linie de intarziere - retine ultimele M valori |y(n)|
.var S;             // media mobila a amplitudinii semnalului de iesire
.var ref;           // nivelul tinta dorit pentru iesire
.var mu;            // pasul de adaptare al castigului

/**/ Tabela de intreruperi ***/
.SECTION/PM interrupts;
```

```

jump start; rti; rti; rti; // 0x00: RESET
jump agc; rti; rti; rti; // 0x04: IRQ2 - esantion nou de la codec
rti; rti; rti; rti; // 0x08: IRQL1
rti; rti; rti; rti; // 0x0C: IRQLO
rti; rti; rti; rti; // 0x10: SPORT0 TX
rti; rti; rti; rti; // 0x14: SPORT0 RX
rti; rti; rti; rti; // 0x18: IRQE
rti; rti; rti; rti; // 0x1C: BDMA
rti; rti; rti; rti; // 0x20: SPORT1 TX / IRQ1
rti; rti; rti; rti; // 0x24: SPORT1 RX / IRQ0
rti; rti; rti; rti; // 0x28: Timer
rti; rti; rti; rti; // 0x2C: Power Down

```

```

.section/PM seg_code;

```

```

start:

```

```

    si = 0;
    dm(S) = si; // initializam media la 0

    ay1 = 0; // g(n-1) fractionar = 0
    ax1 = 0; // g(n-1) intreg = 0 => castig initial = 0

```

```

    si = 0.5r;
    dm(ref) = si; // nivelul = 0.5 (jumatate din maxim)

```

```

    si = 0.05r;
    dm(mu) = si; // pas de adaptare mic = reactie lenta si stabila

```

```

    se = -K; // pentru shiftul aritmetic de la pasul de scalare

```

```

    l3 = M; // lungimea bufferului circular delay[]
    i3 = delay; // pointer la delay[]
    m3 = 1; // pas de incrementare al pointerului

```

```

    imask = 0x200; // activeaza intreruperea IRQ2 (bitul 9)

```

```

stop:

```

```

    nop;
    jump stop; // asteptam intreruperi, procesarea e in agc:

```

```
// ---- Rutina AGC ----
```

```
agc:
```

```
// a) Calcul  $y(n) = g(n-1) * x(n)$ 
```

```
call read_input; // citeste esantionul curent x(n)
```

```
mx0 = ar; // salvam x(n) in MX0
```

```
mr = 0;
```

```
my0 = -1.0r; // MY0 = -1.0, folosit pentru a aduna termenul intreg
```

```
ar = ax1 - 1;
```

```
if lt jump g_fr; // daca ax1=0 (castig < 1.0) sarim peste termenul intreg
```

```
mr = mr - mx0 * my0 (ss); // mr = 0 - x(n)*(-1) = +x(n), adica ax1*x(n) cu ax1=1
```

```
g_fr:
```

```
my1 = ay1;
```

```
mr = mr + mx0 * my1 (rnd); // mr1 = y(n) = g(n-1) * x(n)
```

```
// g reprezentat ca: g = ax1(intreg) + ay1(fractionar)
```

```
dm(tx_buf + 2) = mr1; // scriem y(n) la iesire
```

```
// b) Medie mobila  $S = (1/M) * \text{SUM}\{ |y(n-k)| \}$ 
```

```
ar = abs mr1; // |y(n)|
```

```
sr = ashift ar (hi); // sr1 = |y(n)| / 4 = |y(n)| / M (scalare pt a evita overflow)
```

```
dm(i3, m3) = sr1; // actualizam linia de intarziere
```

```
cntr = M - 1;
```

```
mr = 0, mx0 = dm(i3, m3); // resetam suma si citim primul element
```

```
do sum until ce;
```

```
sum: mr = mr - mx0 * my0(ss), mx0 = dm(i3, m3); // mr += element, citim urmatorul
```

```
mr = mr - mx0 * my0 (rnd); // adaugam ultimul element
```

```
if mv sat mr; // saturare la overflow
```

```
dm(S) = mr1; // S = suma / M (media, datorita scalarii anterioare)
```

```
// c) Actualizare castig  $g(n) = g(n-1) + \mu * (\text{ref} - S)$ 
```

```

ax0 = dm(ref);
ay0 = dm(S);
ar = ax0 - ay0;          // eroarea: ref - S
                        // pozitiva => castigul creste, negativa => scade

my1 = ar;
mx1 = dm(mu);
mr = mx1 * my1 (ss);    // mu * (ref - S)

ar = mr1 + ay1;         // noua parte fractionara a castigului g(n)

af = pass ar;
if ge jump cont;        // daca >= 0 e ok, salvam direct

// d) Corectie daca castigul a depasit domeniul reprezentabil
af = ax1 - 1;
if lt jump cor1;        // ax1=0: castigul trece de 1.0 => il facem 1.0

// ax1=1: castigul ar depasi ~2.0 => saturare la maxim (~1.9999)
ax1 = 0;
ay1 = 0x7fff;
rti;

cor1:
// ax1=0 => castigul tocmai a trecut de 0.9999, il promovam la 1.0
ax1 = 1;
ay1 = 0;
rti;

cont:
ay1 = ar; // salvam noua valoare fractionara a castigului
rti;

// ---- Subrutine I/O ----
read_input:
ar = dm(rx_buf + 2); // citim esantionul de intrare (canal drept)
rts;

write_output:
dm(tx_buf + 2) = mr1; // scriem esantionul de iesire (canal drept)

```

```
rts;
```

➤ Adaptive Line Enhancer – ALE

○ ALE_IO:

```
.section/pm program; // codul se plasează în memoria de program
```

```
.global read_input; // face read_input vizibilă global
```

```
.global write_output; // face write_output vizibilă global
```

```
.extern rx_buf; // buffer de recepție definit în alt fișier (ALE_TEST.DSP)
```

```
.extern tx_buf; // buffer de transmisie definit în alt fișier
```

```
read_input:
```

```
ar = dm(rx_buf+2);
```

```
// Preia eșantionul audio de la codec din bufferul de recepție SPORT
```

```
// rx_buf are 3 cuvinte (structură codec); cuvântul la offset +2
```

```
// este eșantionul audio util (celelalte două sunt date de control/stare)
```

```
// Rezultatul ajunge în registrul ar
```

```
rts; // Returnează controlul; apelantul va citi ar pentru eșantionul primit
```

```
write_output:
```

```
dm(tx_buf+2) = mr1;
```

```
// Stochează rezultatul filtrului (mr1, ieșirea FIR) în bufferul de transmisie
```

```
// la offset +2 — același câmp audio, dar pentru transmisie spre codec
```

```
rts; // Întoarce controlul
```

○ ALE_FCT:

```
/* Acest program implementează rutinele de filtrare FIR și de adaptare  
a coeficienților conform algoritmului LLMS (Leaky LMS).
```

```
*/
```

```
.section/pm program; // codul este plasat în memoria de program (PM)
```

```
.global fir; // exportă funcția 'fir' pentru a fi accesibilă din alte fișiere
```

```

.global llms;      // exportă funcția 'llms' pentru a fi accesibilă din alte fișiere

//
=====

// Subrutina FIR — calculează rezultatul filtrului FIR
//
=====

/*
Parametri necesari înainte de apelul fir:
i3 = pointer circular către linia de întârziere (eșantioanele de intrare)
i6 = pointer circular către vectorul de coeficienți
m1 = 1 (increment pentru pointerul i3)
m5 = 1 (increment pentru pointerul i6)
cntr = N-1 (numărul de pași ai buclei)
Rezultat returnat:
mr1 = suma produselor  $x[n-k] * h[k]$  (ieșirea filtrului)
*/

fir:  mr=0, mx0=dm(i3,m1), my0=pm(i6,m5);
      // Resetează acumulatorul mr la zero
      // Încarcă primul eșantion din linia de întârziere în mx0 (din data memory)
      // Încarcă primul coeficient în my0 (din program memory)
      // (accesele duble în paralel sunt specifice arhitecturii ADSP-2181)

do sop until ce;    // execută bucla de 'cntr' ori (cntr setat la N-1)

sop:  mr=mr+mx0*my0(ss), mx0=dm(i3,m1), my0=pm(i6,m5);
      // Adună produsul mx0*my0 în acumulatorul mr (înmulțire signed*signed)
      // Concomitent: preia următorul eșantion și următorul coeficient

      mr=mr+mx0*my0(rnd);
      // Încorporează ultimul produs și aplică rotunjirea rezultatului

      if mv sat mr;
      // Verifică flagul de overflow (mv = multiply overflow)
      // Dacă e setat, saturează mr la limita maximă/minimă (previne wrap-around)

```

```
//
=====
===
// Subrutina LLMS — Leaky LMS — modifică coeficienții filtrului FIR
//
=====
===
/*
Formula: h(k) = h(k)*(1 - mu*lambda) + mu * x(n-k) * e(n)
          ^^^^^^^^^^^^^^^^^^^^^      ^^^^^^^^^^^^^^^^^^^^^
          componenta "leaky"    componenta LMS clasică
```

```
mx0 = semnalul de eroare e(n)
my1 = mu (rata de adaptare)
mx1 = lambda (coeficientul de "scurgere" / leakage)
i3 = pointer circular către linia de întârziere x(n-k)
i6 = pointer circular către coeficienții h(k)
m1=1, m5=1 (incrementare standard)
m4=-1      (decrementare — citire fără avans)
m6=2      (avans cu 2 — salt după scriere)
m3=-1     (corecție finală pointer)
m7=0      (acces fără modificarea pointerului)
cntr = N   (numărul total de coeficienți)
*/
```

```

lms:  mr=mx0*my1(rnd), mx0=dm(i3,m1);
      // Calculează produsul  $\mu * e(n) \rightarrow mr1$ 
      // Concomitent: preia primul eșantion  $x(n-k)$  în mx0

      my0=mr1;
      // Transferă rezultatul  $\mu * e(n)$  în my0 pentru utilizare ulterioară

      mr=mx0*my0(rnd), ay0=pm(i6,m5);
      // Calculează  $x(n-k) * \mu * e(n) \rightarrow mr1$  (termenul de actualizare LMS)
      // Concomitent: preia coeficientul  $h(k)$  curent în ay0

```


do adaptive1 until ce; // parcurge toți coeficienții filtrului

```
ar=mr1+ay0, mx0=dm(i3,m1), ay0=pm(i6,m4);  
// ar = h(k) + x(n-k)*mu*e(n) — aplică corecția LMS asupra coeficientului  
// Concomitent: preia eșantionul următor și recitește h(k) cu pointer decrementat
```

```
mf=mx1*my1(rnd), sr1=pm(i6,m7);  
// mf = lambda * mu (produsul factorilor leaky)  
// sr1 = valoarea curentă h(k) (citire fără modificarea pointerului)
```

```
mr=sr1*mf(rnd);  
// mr1 = h(k) * mu * lambda (componenta de atenuare "leaky")
```

```
ay1=mr1;  
// Stochează termenul leaky în ay1 pentru pasul următor
```

```
ar=ar-ay1;  
// ar = h(k) + x(n-k)*mu*e(n) - h(k)*mu*lambda  
// = h(k)*(1 - mu*lambda) + mu*x(n-k)*e(n)  
// → rezultatul complet al formulei LLMS
```

```
adaptive1: pm(i6,m6)=ar, mr=mx0*my0(rnd);  
// Stochează coeficientul actualizat h(k) în memoria PM  
// Concomitent: precalculează x(n-k)*mu*e(n) pentru iterația viitoare
```

```
modify(i3,m3);  
// Repoziționează pointerul i3 cu m3=-1 după finalizarea buclei
```

```
modify(i6,m4);  
// Repoziționează pointerul i6 cu m4=-1 după finalizarea buclei
```

rts; // Redă controlul funcției apelante

○ ALE_TEST:

```
/******
```

```
*
```

```
* Acest program testeaza filtrul ALE :
```

```
*
```

```
* - tabelul de intreruperi specific ADSP2181
```

* - rutinele de servire a intreruperilor

*****/

```
.extern fir;      // filtrul FIR (din ALE_FCT.DSP)
.extern lms;      // LMS clasic (nefolosit aici, dar declarat)
.extern llms;     // Leaky LMS (din ALE_FCT.DSP)
.extern read_input; // citire codec (din ALE_IO.DSP)
.extern write_output; // scriere codec (din ALE_IO.DSP)

#include "ale.h"    // include constantele N, delay, mu, lambda

// -----
// Alocarea memoriei — secțiunea de date
// -----
.section/dm data1;

.var/circ fir_d[n]; // linia de întârziere a FIR — buffer circular de N eșantioane
                    // conține eșantioanele  $x(n-k)$  utilizate la filtrare
.var/circ input[n]; // linia de întârziere de intrare — buffer circular de N eșantioane
                    // stochează semnalul brut de intrare (cu zgomot)

.section/pm data2;

.var/circ hh[n];    // coeficienții FIR  $h(k)$  — buffer circular de N valori
                    // inițializați cu 0, modificați de LLMS

.section/dm data1;

.var/circ rx_buf[3]; // buffer circular de 3 cuvinte pentru recepție SPORT (codec →
DSP)
.var/circ tx_buf[3]; // buffer circular de 3 cuvinte pentru transmisie SPORT (DSP →
codec)
.global rx_buf;      // accesibil din ALE_IO.DSP
.global tx_buf;      // accesibil din ALE_IO.DSP
.var save;           // variabilă temporară pentru păstrarea eșantionului curent

//
=====
===
// Tabela vectorilor de întrerupere — obligatorie pentru ADSP-2181
```

// Fiecare slot are 4 instrucțiuni (16 bytes)

//

=====

====

.section/pm IVreset;

jump start; rti; rti; rti; // 0x00: Reset — sare la inițializare

jump input_samples; rti; rti; rti; // 0x04: IRQ2 — întreruperea de eșantionare

rti; rti; rti; rti; // 0x08: IRQL1 — nefolosit

rti; rti; rti; rti; // 0x0C: IRQL0 — nefolosit

rti; rti; rti; rti; // 0x10: SPORT0 TX — nefolosit

rti; rti; rti; rti; // 0x14: SPORT1 RX — nefolosit

rti; rti; rti; rti; // 0x18: IRQE — nefolosit

rti; rti; rti; rti; // 0x1C: BDMA — nefolosit

rti; rti; rti; rti; // 0x20: SPORT1 TX sau IRQ1 — nefolosit

rti; rti; rti; rti; // 0x24: SPORT1 RX sau IRQ0 — nefolosit

rti; rti; rti; rti; // 0x28: Timer — nefolosit

rti; rti; rti; rti; // 0x2C: Power Down — nefolosit

//

=====

====

// Rutina de inițializare — executată o singură dată la reset

//

=====

====

.section/pm program;

start:

m1=1; // increment pentru pointerii pe DM

m5=1; // increment pentru pointerii pe PM

si=0; // registru sursă = 0 (pentru aducerea la 0 a bufferelor)

i2=input; // i2 indică începutul buffer-ului de intrare

l2=n; // l2 = dimensiunea bufferului (adresare circulară)

i4=input+delay; // i4 indică intrarea decalată cu 'delay' eșantioane

l4=n; // acesta este semnalul $x(n-\text{delay})$ folosit în FIR

// 'delay' este definit în ale.h — separarea corelației

i3=fir_d; // i3 indică linia de întârziere a FIR

l3=n;

i6=hh; // i6 indică vectorul de coeficienți FIR

l6=n;

```

// aduce la 0 toate bufferele
    cntr=n;
    do init_buf until ce;
    dm(i2,m1)=si; // scrie 0 în linia de întârziere de intrare
    dm(i3,m1)=si; // scrie 0 în linia de întârziere FIR
init_buf:
    pm(i6,m5)=si; // scrie 0 în coeficienți (filtrul pornește "neutral")

    imask=0x200; // activează întreruperea IRQ2 (bitul 9 din masca de întreruperi)
                // de acum DSP-ul va răspunde la semnalul de eșantionare

//
=====

===
// Bucla principală — așteaptă întreruperi
//
=====

===
wait:  jump wait;    // buclă infinită — tot procesul se desfășoară în ISR

//
=====

===
// ISR — Rutina de servire a întreruperii IRQ2 (sosit eșantion nou)
// Aceasta este nucleul algoritmului ALE, rulată la fiecare eșantion
//
=====

===
input_samples:
    ena sec_reg;      // comută pe setul secundar de registre
                      // protejează registrele principale fără salvare pe stivă
    ena ar_sat;       // activează saturarea pentru registrul ar
                      // previne overflow la adunări/scăderi

    // --- Pasul 1: Citește eșantionul nou de la codec ---
    call read_input;  // ar = x(n) — eșantionul curent cu zgomot
    dm(save)=ar;      // păstrează x(n) pentru calculul ulterior al erorii

    // --- Pasul 2: Actualizează liniile de întârziere ---

```

```

dm(i2,m1)=ar;          // inserează x(n) în buffer-ul circular 'input'
ar=dm(i4,m5);          // preia x(n-delay) din buffer (eșantionul intarziat)
dm(i3,m1)=ar;          // inserează x(n-delay) în linia de întârziere a FIR
                        // esența ALE: filtrul operează pe semnalul DECATAT
                        // astfel zgomotul (decorelat) nu poate fi estimat

// --- Pasul 3: Calculează ieșirea filtrului FIR ---
cntr=n-1;              // setează numărul de iterații (N-1 în buclă + 1 final)
call fir;              //  $mr1 = \hat{y}(n) = \sum h(k) * x(n-delay-k)$  — estimarea semnalului util

// --- Pasul 4: Transmite ieșirea la codec ---
call write_output;     // trimite mr1 (semnalul "curățat") spre codec/ARM

// --- Pasul 5: Calculează eroarea ---
ar=dm(save);           // reîncarcă x(n) din memorie
ay0=ar;
ar=ay0-mr1;            // eroarea  $e(n) = x(n) - \hat{y}(n)$ 

// --- Pasul 6: Adaptează coeficienții FIR ---
mx0=ar;                //  $mx0 = e(n)$  — transmis către LLMS
cntr=n;
my1=mu;                // pasul de adaptare
m4=-1;                 // decrementare pointer
m6=2;                  // salt după scriere
m3=-1;                 // corecție finală pointer
//call lms;            // LMS clasic — dezactivat, se folosește LLMS
m7=0;                  // citire fără modificarea pointerului
mx1=lambda;            // factorul de scurgere (leakage)
call llms;              // actualizează h(k) conform formulei Leaky LMS

dis sec_reg;           // revine la setul primar de registre
rti;                   // încheie rutina de întrerupere

```

➤ Median Filter – MF

```

#include "def2181.h"    // definitii registre si periferice pentru ADSP-2181
#define W 3             // W = marimea ferestrei filtrului median

```

```

                //fereastra contine W=3 esantioane consecutive
#define K 1      // K = indexul medianei in sirul ordonat (incepe de la 0)
                // pentru W=3 elemente ordonate: [min, MEDIAN, max] --> mereu la
//mijloc

.SECTION/DM data1;
//Datele de mai jos sunt stocate in memoria de date (DM)

.var/circ rx_buf[3]; // buffer circular pentru intrarea audio (receptie de la codec)
.var/circ tx_buf[3]; // buffer circular pentru iesirea audio (transmisie catre codec)
.var/circ delay[W]; // linie de intarziere circulara cu W elemente
                // pastreaza cele mai recente W=3 esantioane primite
                // cand se adauga al 4-lea, primul este suprascris automat

.var delay_sorted[W]; // buffer LINEAR (nu circular) cu W elemente
                // primeste o copie a esantioanelor din delay[] pentru sortare
                // delay[] nu e sortat direct, pentru a pastra ordinea temporala
                // folosit exclusiv pentru determinarea medianei

/**** Tabelul Vectorilor de Intrerupere ****/
.SECTION/PM interrupts;
// Tabel cu adrese fixe in memoria de program
// La aparitia unui eveniment, procesorul sare automat la adresa corespunzatoare
// Fiecare slot ocupa 4 locatii (jump + 3x rti)

jump start; rti; rti; rti; /*00: RESET - la pornire, sare la rutina de initializare */
jump median_f; rti; rti; rti; /*04: IRQ2 - semnal nou de la codec, apeleaza filtrul
median */
rti; rti; rti; rti; /*08: IRQL1 - neutilizata */
rti; rti; rti; rti; /*0c: IRQL0 - neutilizata */
rti; rti; rti; rti; /*10: SPORT0 tx - neutilizata */
rti; rti; rti; rti; /*14: SPORT1 rx - neutilizata */
rti; rti; rti; rti; /*18: IRQE - neutilizata */
rti; rti; rti; rti; /*1c: BDMA - neutilizata */
rti; rti; rti; rti; /*20: SPORT1 tx sau IRQ1 - neutilizata */
rti; rti; rti; rti; /*24: SPORT1 rx sau IRQ0 - neutilizata */
rti; rti; rti; rti; /*28: timer - neutilizata */
rti; rti; rti; rti; /*2c: power down - neutilizata */
// rti="Return from Interrupt" este echivalentul unei instructiuni goale
// intreruperile nefolosite sunt completate cu rti pentru a evita executia de cod aleator

```

```

.section/PM seg_code;
// Codul programului in memoria de program

/*****
*****
* Initializare ADSP 2181
*****
*****/

start:
// Aceasta sectiune se executa O SINGURA DATA la pornirea sistemului

l3=W; // seteaza lungimea bufferului circular pentru i3 = W
      // fara aceasta setare, i3 nu stie unde sa se intoarca
      // la capatul bufferului delay[]
i3=delay; // i3 pointeaza la primul element al bufferului delay[]
          // i3 este folosit pentru citiri/scrieri in delay
m3=1; // modificadorul lui i3 = 1
      // dm(i3, m3) = acceseaza adresa i3, apoi incrementeaza i3 cu 1
l0=0; // l0=0 -> i0 nu functioneaza circular
      // i0 va fi folosit pentru delay_sorted (buffer liniar)
      // daca l0 ar fi nenul, i0 s-ar comporta circular
      // si ar produce citiri/scrieri incorecte in delay_sorted

imask=0x200; // activeaza intreruperea IRQ2 (bitul 9 din registrul IMASK)
             // de acum, la fiecare esantion nou de la codec
             // se va declansa automat rutina median_f

stop:
nop; // eticheta "stop" plasata pe o instructiune NOP (inactiva)
jump stop; // bucla infinita - procesorul stationeaza aici
           // toata procesarea utila se desfasoara in rutinele de intrerupere (median_f)

/*****
*****
* RUTINA PRINCIPALA: median_f
* Rulează la fiecare esantion nou de la codec (declansata de IRQ2)

```

*****/

median_f:

// --- PASUL 1: Citeste esantionul nou si actualizeaza fereastra ---

call read_input; // apeleaza subrutina read_input

// rezultat: AR = x(n) = esantionul curent de la codec

dm(i3,m3)=ar; // stocheaza x(n) in delay[i3] si avanseaza i3 cu 1

// deoarece delay[] e circular cu l3=W,

// i3 revine automat la inceput dupa 3 scrieri

// -> delay[] contine mereu cele mai recente W=3 esantioane

// --- PASUL 2: Copiaza fereastra delay[] in delay_sorted[] ---

// Sortarea nu se face direct in delay[] pentru a nu altera ordinea temporală

i0=delay_sorted; // la inceputul lui delay_sorted[] avem pointerul i0

// aici se va scrie copia pentru sortare

cntr=W; // contor = W = 3 (numarul de elemente de copiat)

do copy until ce; // bucla: se repeta pana contorul ajunge la 0

si=dm(i3,m3); // citeste un element din delay[] si avanseaza i3

// dupa W citiri, i3 a traversat complet delay[] si e inapoi la start

copy:

dm(i0,m3)=si; // scrie elementul citit in delay_sorted[] si avanseaza i0

// rezultat: delay_sorted[] = copie a lui delay[]

// --- PASUL 3: Sorteaza delay_sorted[] prin selectie ---

i2=delay_sorted; // i2 = pointer de baza la array-ul ce urmeaza a fi sortat

// sort_sel foloseste i2 pentru a naviga in array

call sort_sel; // apeleaza subrutina de sortare prin selectie

// dupa revenire: delay_sorted[] este ordonat crescator

// --- PASUL 4: Extrage mediana si transmite la iesire ---

i0=delay_sorted; // reseteaza i0 la inceputul lui delay_sorted[]

// dupa sortare, i0 se afla undeva in interiorul array-ului

m0=K; // m0 = K = 1 (pozitia medianei in sirul ordonat)

modify(i0,m0); // i0 = i0 + K -> muta pointerul cu K=1 pozitii fara sa citeasca

// acum i0 e pointer exact la elementul median

ar=dm(i0,m3); // citeste mediana din delay_sorted[K] -> AR

// ex: din [-500, 100, 8000], se citeste 100

dm(tx_buf+2)=ar; // trimite mediana in bufferul de iesire catre codec


```

        // tx_buf+2 = adresa elementului de iesire audio
rti;      // Return from Interrupt - revine la bucla stop: jump stop

/*****
*****
* SUBROUTINA: sort_sel
* Sorteaza delay_sorted[] folosind algoritmul de sortare prin selectie
* i2 = pointer de baza la array (transmis de apelant)
*
* Echivalent C:
* for (i=0; i<W; i++) {
*   min=i;
*   for (j=i+1; j<W; j++) {
*     if (A[j] < A[min]) min=j;
*   }
*   tmp=A[i]; A[i]=A[min]; A[min]=tmp;
* }

*****
*****/

sort_sel:
// Utilizarea registrilor modificatori in aceasta subrutina:
// m0 = pozitia elementului minim identificat pana acum
// m1 = pozitia elementului de la care porneste iteratia (i)
// m2 = pozitia de start a cautarii interioare (i+1)
// i2 = pointer fix la baza lui delay_sorted[] (nu se modifica)
// i0, i1 = pointeri temporari pentru accesarea A[j] si A[min]

m3=1;    // pasul de incrementare pentru citiri/scrieri = 1
i0=0;    // i0=0 -> i0 nu e circular (pointer liniar in delay_sorted)
i1=0;    // i1=0 -> i1 nu e circular (la fel)
cntr=W;  // contor bucla exterioara = W = 3

// === BUCLA EXTERIOARA (loop_i): parcurge fiecare pozitie i ===
do loop_i until ce;

// Calculeaza valoarea lui i din contorul descrescator
// cntr scade de la W la 1, deci i = W - cntr
ax0=W;   // ax0 = W = 3 (valoare constanta)
ay0=cntr; // ay0 = valoarea curenta a contorului

```

```

ar=ax0-ay0;    // ar = W - cntr = i (iteratia curenta: 0, 1, 2)
m0=ar;        // m0 = i -> minimul este presupus initial la pozitia i
m1=ar;        // m1 = i -> retinem pozitia lui i pentru schimbarea de la final
ar=ar+1;      // ar = i + 1
m2=ar;        // m2 = i+1 -> de aici incepe cautarea minimului

ay0=m1;       // ay0 = i
ar=ax0-ay0;   // ar = W - i = numarul de elemente de la i pana la sfarsit
ar=ar-1;      // ar = W - i - 1 = numarul de iteratii ale buclei interioare
               // ex: i=0 -> ar=2 (comparam A[1] si A[2] cu A[min])
               // i=1 -> ar=1 (comparam A[2] cu A[min])
               // i=2 -> ar=0 (nu mai sunt elemente de comparat)

if eq jump loop_i; // daca ar=0 (ultima iteratie), nu mai exista bucla interioara
               // sarim direct la sfarsitul buclei exterioare

cntr=ar;      // actualizeaza contorul buclei interioare

// Pozitioneaza i0 la A[i+1]
i0=i2;        // i0 = baza array-ului
modify(i0,m2); // i0 = delay_sorted + (i+1) -> pointer la A[i+1]

// === BUCLA INTERIOARA (loop_j): identifica minimul in A[i+1..W-1] ===
do loop_j until ce;

ax0=dm(i0,m3); // ax0 = A[j] (citeste elementul curent si avanseaza i0)
i1=i2;
modify(i1,m0); // i1 pointer la A[min]
ay0=dm(i1,m3); // ay0 = A[min] (citeste minimul curent)
ar=ax0-ay0;    // ar = A[j] - A[min]
if gt jump loop_j; // daca A[j] > A[min]: nu actualiza, treci la urmatorul j

// A[j] <= A[min]: s-a gasit un nou minim la pozitia j
ax0=W;
ay0=cntr;
ar=ax0-ay0;    // ar = j (pozitia curenta calculata din contor)
m0=ar;        // m0 = j -> retine noua pozitie a minimului

loop_j: nop;    // ultima instructiune a buclei interioare
               // nop previne hazarduri de pipeline

```

```

// === INTERSCHIMBARE: A[i] <-> A[min] ===
i1=i2;
modify(i1,m1); // i1 -> A[i]
ax1=dm(i1,m3); // ax1 = valoarea de la pozitia i (temporar)

i1=i2;
modify(i1,m0); // i1 -> A[min]
ay1=dm(i1,m3); // ay1 = valoarea de la pozitia min

i1=i2;
modify(i1,m1); // i1 -> A[i] din nou
dm(i1,m3)=ay1; // A[i] = A[min]

i1=i2;
modify(i1,m0); // i1 -> A[min] din nou
dm(i1,m3)=ax1; // A[min] = vechea valoare de la i
// interschimbarea e completa

loop_i: nop; // ultima instructiune a buclei exterioare
// nop previne hazarduri de pipeline

// Dupa W iteratii, delay_sorted[] este complet ordonat crescator
rts; // Return from Subroutine - revine in median_f

////////////////////////////////////
// Functii de intrare si iesire
////////////////////////////////////
read_input:
ar=dm(rx_buf+2); // preia esantionul curent din bufferul de receptie al codec-ului
// rx_buf+2 = adresa elementului de intrare audio
// valoarea e plasata in AR si returnata apelantului
rts; // Return from Subroutine

write_output: // functie definita dar neapelata explicit in cod
dm(tx_buf+2)=mr1; // scrie MR1 in bufferul de transmisie
// in median_f scriem direct AR, nu prin aceasta functie
rts; // Return from Subroutine

```

9. Cod – VisualDSP++

```
#include "def2181.h"      // definitii pentru registrele/perifericele ADSP-2181

#define PORT_OUT 0xFF      // adresa extensiei de iesire (afisor / leduri)
#define PORT_IN 0x1FF      // adresa extensiei de intrare (switch-uri)

#define AGC_M 4           // numarul de esantioane pentru media AGC
#define AGC_K 2           // factor de scalare pentru |y(n)|
#define ALE_N 8           // ordinul filtrului FIR adaptiv
#define ALE_DELAY 2       // intarzierea folosita de ALE
#define MF_W 3            // dimensiunea ferestrei pentru median filter W=2k+1
#define MF_K 1            // pozitia medianei in sirul sortat pentru W=3

/* Comenzi de simulare:
 * 0x00C0 = AGC stanga
 * 0x00C1 = ALE stanga
 * 0x00C2 = MF stanga
 * 0x00C8 = AGC dreapta
 * 0x00C9 = ALE dreapta
 * 0x00CA = MF dreapta
 */
#define SIM_CMD 0x00C8     // comanda fixa folosita in simulare; aici testeaza
                          // AGC

/*=====
 * BUFERE CODEC
 *=====*/
.section/dm buf_var1;      // sectiune DM pentru buffer de receptie
.var rx_buf[3];           // buffer circular de receptie de la codec

.section/dm buf_var2;      // sectiune DM pentru buffer de transmisie
.var tx_buf[3] = 0xc000, 0x0000, 0x0000; // buffer initializat pentru codec

.section/dm buf_var3;      // sectiune DM pentru comenzile de initializare
                          // codec
.var init_cmds[13] = 0xc002, 0xc102, 0xc288, 0xc388, 0xc488, 0xc588,
                    0xc680, 0xc780, 0xc85c, 0xc909, 0xca00, 0xcc40, 0xcd00; // secventa
                          // de setup codec
```

```

/*=====
*  VARIABLE SI BUFERE
*=====*/
.SECTION/DM data1;

.var stat_flag;          // flag folosit la initializarea codec-ului( daca este egal cu 1,
                          // initializarea este in curs, daca este 0, initializarea a fost făcuta

.var PF_input;           // ultima valoare citita de pe portul PF
.var PF_output;          // rezervat pentru iesire PF
.var Flag_CDA;           // flag care semnaleaza comanda noua de la ARM
.var cmd_comanda;        // comanda pe 8 biti primită de la ARM
.var cmd_corect;         // 1 daca D7=D6=1, altfel 0
.var id_functie;         // 0=AGC, 1=ALE, 2=MF, selectat din D2-D0
.var param_id;           // parametru selectat din D5-D4
.var canal;              // canalul audio selectat din D3
.var x_in;               // esantionul curent de intrare de la codec
.var y_out;              // esantionul curent de iesire spre codec

.var sw_value;           // valoarea switch-urilor de pe extensie
.var disp_value;         // valoarea trimisa pe afisor 7 segmente

/* AGC */
.var agc_delay[AGC_M];   // linia de intarziere |y(n-k)|, k=0..M-1
.var agc_S;              // media |y(k)|
.var agc_ref;            // nivelul de referinta dorit REF
.var agc_mu;             // pasul de adaptare
.var agc_g_int;          // partea intreaga a castigului
.var agc_g_frac;         // partea fractionara a castigului

/* ALE */
.var ale_input[ALE_N];   // linia de intrare x(n), x(n-1), ...
.var ale_fir_d[ALE_N];   // linia FIR (intrari intarziate cu D)
.var ale_save;           // salveaza x(n) pentru calculul erorii
.var ale_mu;             // pasul de adaptare pentru ALE
.var ale_lambda;         // factorul de leakage

/* MF */
.var mf_delay[MF_W];     // linia de intarziere circulara (W esantioane)

```

```

.var mf_delay_sorted[MF_W]; // copie liniara pentru sortare
.var mf_K; // indexul medianei

/*=====
* COEFICIENTI ALE in PM
*=====*/

.SECTION/PM pm_da; // sectiune PM pentru date in memorie de program
.var ale_hh[ALE_N]; // coeficientii filtrului adaptiv ALE

/*=====
* VECTORI DE INTRERUPERE
*=====*/

.SECTION/PM interrupts; // tabelul vectorilor de intrerupere
    jump start; rti; rti; rti; // vector reset
    rti; rti; rti; rti; // IRQ2
    rti; rti; rti; rti; // IRQL1
    rti; rti; rti; rti; // IRQL0

    ar = dm(stat_flag); // la SPORT0 TX verificam daca mai avem
comenzi init
    ar = pass ar; // copiem pentru test
    if eq rti; // daca stat_flag=0, iesim din intrerupere
    jump next_cmd; // altfel trimitem urmatoarea comanda codec

    jump input_samples; // SPORT0 RX: tratam esantionul primit
    rti; rti;

    jump isr_ia; rti; rti; rti; // IRQE: semnal de la ARM

    rti; rti; rti; rti; // BDMA
    rti; rti; rti; rti; // SPORT1 TX/IRQ1
    rti; rti; rti; rti; // SPORT1 RX/IRQ0
    jump isr_periodic; rti; rti; rti; // timer: masina de stari
    rti; rti; rti; rti; // power down

.SECTION/PM seg_code; // sectiunea principala de cod

```

```

/*****
* START
*****/

start:
    ax0 = b#0000100000000000;    // configurare initiala sistem
    dm(Sys_Ctrl_Reg) = ax0;        // scrie valoarea din ax0 în registrul Sys_Ctrl_Reg
    ena timer;                     // activeaza timerul

    i5 = rx_buf;                   // i5 pointeaza la bufferul de receptie
    l5 = LENGTH(rx_buf);           // lungimea bufferului circular rx_buf
    i6 = tx_buf;                   // i6 pointeaza la bufferul de transmisie
    l6 = LENGTH(tx_buf);           // lungimea bufferului circular tx_buf
    i3 = init_cmds;                // i3 pointeaza la comenzile de init codec
    l3 = LENGTH(init_cmds);        // lungimea sirului init_cmds

    m1 = 1;                        // increment standard cu 1
    m5 = 1;

/* *****SPORT0 ***** */
    ax0 = b#0000110011010111;    // configurare autobuffering SPORT0
    dm(Sport0_Autobuf_Ctrl) = ax0; // scriere configuratie SPORT0

    ax0 = 0;                       // divizor frame sync = 0
    dm(Sport0_Rfsdiv) = ax0;       // scrie RFS divider
    ax0 = 0;                       // divizor clock = 0
    dm(Sport0_Sclkdiv) = ax0;      // scrie SCLK divider

    ax0 = b#1000011000001111;    // configurare SPORT0 control register
    dm(Sport0_Ctrl_Reg) = ax0;     // activeaza SPORT0

    ax0 = b#0000000000000111;    // 3 cuvinte pe tx slot 0
    dm(Sport0_Tx_Words0) = ax0;    // seteaza nr cuvinte tx0
    ax0 = b#0000000000000111;    // 3 cuvinte pe tx slot 1
    dm(Sport0_Tx_Words1) = ax0;    // seteaza nr cuvinte tx1
    ax0 = b#0000000000000111;    // 3 cuvinte pe rx slot 0
    dm(Sport0_Rx_Words0) = ax0;    // seteaza nr cuvinte rx0
    ax0 = b#0000000000000111;    // 3 cuvinte pe rx slot 1
    dm(Sport0_Rx_Words1) = ax0;    // seteaza nr cuvinte rx1

/* SYSTEM */
    ax0 = b#0001100000000000;    // configurare finala sistem

```

```

dm(Sys_Ctrl_Reg) = ax0;    // scriere registru control sistem

ifc = b#00000011111110;    // stergere eventuale intreruperi pendinte
nop;                        // pauza pentru stabilizare

icntl = b#00010;           // configurare control intreruperi
mstat = b#1100000;         // mod fractional / alte setari DSP

/* INITIALIZARE CODEC */
ax0 = 1;                   // 1 = mai avem de trimis comenzi de init
dm(stat_flag) = ax0;       // salveaza flag init

ena ints;                  // activeaza intreruperile globale
imask = b#0001000001;      // permite SPORT0 TX + alte intreruperi
necesare init

ax0 = dm(i6, m5);          // citeste primul cuvant din tx buffer
tx0 = ax0;                 // il trimite pe SPORT0 pentru pornire init

check_init:
ax0 = dm(stat_flag);       // citeste daca init s-a terminat
af = pass ax0;             // copie pentru test
if ne jump check_init;     // daca nu s-a terminat, asteapta

ay0 = 2;                   // masca pentru bit ACI
check_aci1:
ax0 = dm(rx_buf);          // citeste cuvantul de status din rx_buf
ar = ax0 and ay0;          // extrage bitul ACI
if eq jump check_aci1;     // asteapta sa devina 1

check_aci2:
ax0 = dm(rx_buf);          // reciteste status
ar = ax0 and ay0;          // verifica acelasi bit
if ne jump check_aci2;     // asteapta sa revina la 0
idle;                      // lasa DSP-ul in idle pana la urmatorul eveniment

//dezactivam mute pe canalul stang de iesire folosind masca 0xbf3f
ay0 = 0xbf3f;
ax0 = dm(init_cmds + 6);
ar = ax0 and ay0;

```



```

dm(tx_buf) = ar;
idle;

// dezactivam mute pe canalul drept

ax0 = dm(init_cmds + 7);
ar = ax0 and ay0;
dm(tx_buf) = ar;
idle;

// curatam flag urile de intreruperi pendinte

ifc = b#00000011111110;
nop;

imask = b#0000100000;      // lasa doar SPORT0 RX activ

/* INITIALIZARI ETAPA 2 */
si = 0xFFFF;              // maxim wait state
dm(Dm_Wait_Reg) = si;      // scrie registrul wait
//Dm_Wait_Reg – Data Memory Wait State Register, un registru hardware care
controleaza cate cicluri de asteptare insereaza DSP-ul cand acceseaza memoria
externă

si = 0x0000;              // PF configurat ca intrare
dm(Prog_Flag_Comp_Sel_Ctrl) = si; // scrie configurarea PF
//Prog_Flag_Comp_Sel_Ctrl - Programmable Flag Complement Select Control,
registru care controleaza directia si comportamentul pinilor PF0-PF7, pe acesti
pini vine comanda de la ARM

ax0 = 0;                  // zero pentru resetarea variabilelor
dm(PF_input)  = ax0;
dm(PF_output) = ax0;
dm(Flag_CDA)  = ax0;
dm(cmd_comanda) = ax0;
dm(cmd_corect) = ax0;
dm(id_functie) = ax0;
dm(x_in)      = ax0;
dm(y_out)     = ax0;
dm(param_id)  = ax0;
dm(canal)     = ax0;

```

```

dm(sw_value) = ax0;      // switch-uri 0
dm(displ_value) = ax0;    // afisaj 0

/***** AGC initializare castig 0, REF=0.5, mu = 0.05 *****/
si = 0;                  // zero
dm(agc_S) = si;          // media AGC initial 0
dm(agc_g_int) = si;      // partea intreaga castig 0
dm(agc_g_frac) = si;     // partea fractionara castig 0
si = 0.5r;              // referinta initiala 0.5
dm(agc_ref) = si;        // scrie ref
si = 0.05r;             // viteza adaptare initiala 0.05
dm(agc_mu) = si;         // scrie mu

i3 = agc_delay;          // pointer spre delay line AGC
l3 = AGC_M;              // lungime circulara = 4
m3 = 1;                  // increment 1
cntr = AGC_M;            // numar elemente de resetat
si = 0;
do clr_agc until ce;     // bucla de reset AGC
clr_agc:
    dm(i3, m3) = si;     // scrie 0 in fiecare element

/***** ALE initializare *****/
i2 = ale_input;          // pointer la buffer intrare ALE
l2 = ALE_N;              // lungime 8
i3 = ale_fir_d;          // pointer la buffer FIR ALE
l3 = ALE_N;              // lungime 8
i6 = ale_hh;             // pointer la coeficienti
l6 = ALE_N;              // lungime 8
cntr = ALE_N;
si = 0;
do clr_ale until ce;     // reset buffere + coeficienti
dm(i2, m1) = si;         // scrie 0 in ale_input
dm(i3, m1) = si;         // scrie 0 in ale_fir_d
clr_ale:
    pm(i6, m5) = si;     // scrie 0 in coeficientii din pm

si = 0.01r;              // mu initial pentru ALE
dm(ale_mu) = si;         // salveaza mu
si = 0.001r;             // lambda initial

```

```

dm(ale_lambda) = si;          // salveaza lambda

/***** MF initializare *****/
i3 = mf_delay;                // pointer la buffer MF
l3 = MF_W;                    // lungime 3
cntr = MF_W;
si = 0;
do clr_mf until ce;           // reset buffer MF
clr_mf:
dm(i3, m3) = si;              // scrie 0 in buffer

ax0 = MF_K;                   // index median initial
dm(mf_K) = ax0;               // salveaza index median

ena ints;                     // activeaza intreruperile
imask = b#1000000110;         // activeaza timer + SPORT0 RX + IRQE

main_loop:
idle;                          // procesorul asteapta intreruperi
jump main_loop;               // bucla infinita

/*****
* ISR_IA – apasare buton IA
*****/
isr_ia:
ena sec_reg;                  // foloseste setul secundar de registre
ax0 = 1;                      // exista comanda noua
dm(Flag_CDA) = ax0;           // seteaza flagul de comanda
rti;                          // intoarcere din intrerupere

/*****
* ISR_PERIODIC – masina de stari
*****/
isr_periodic:
ena sec_reg;                  // comuta pe registrele secundare

starea_0:
ax0 = dm(Flag_CDA);           // citeste flagul de comanda noua
af = pass ax0;                // copie pentru test

```

```
if ne jump starea_1;      // daca avem comanda noua, mergem in starea 1
jump starea_2;           // altfel mergem in starea 2
```

starea_1:

```
ax0 = dm(Prog_Flag_Data); // citeste portul PF7-0
ay0 = 0x00FF;             // masca pe 8 biti
ar = ax0 and ay0;         // retine doar byte-ul util
dm(cmd_comanda) = ar;     // salveaza comanda
```

```
ax0 = 0;                  // comanda a fost preluata
dm(Flag_CDA) = ax0;       // reseteaza flagul
rti;                      // iesire
```

starea_2:

```
ax0 = dm(cmd_comanda);    // citeste comanda curenta
ay0 = 0x00C0;             // masca pentru D7 si D6
ar = ax0 and ay0;         // pastreaza bitii de validare
af = ar - ay0;            // compara cu 0xC0
if ne jump cmd_invalida;  // daca nu sunt ambii 1, bypass
```

```
ax0 = 1;                  // comanda valida
dm(cmd_corect) = ax0;     // memoreaza validitatea
```

//extragere id functie

```
ax0 = dm(cmd_comanda);    // reciteste comanda
ay0 = 0x0007;             // masca D2-D0
ar = ax0 and ay0;         // extrage ID functie
dm(id_functie) = ar;      // salveaza ID
```

//extragere parametru

```
ax0 = dm(cmd_comanda);    // reciteste comanda
ay0 = 0x0030;             // masca D5-D4
ar = ax0 and ay0;         // extrage parametrul codificat
si = ar;                  // muta in SI pentru shift
sr = lshift si by -4 (lo);
dm(param_id) = sr0;       // salveaza param_id
```

//extragere canal

```
ax0 = dm(cmd_comanda);    // reciteste comanda
ay0 = 0x0008;             // masca D3
ar = ax0 and ay0;         // extrage canalul
```

```

si = ar;          // muta in SI
sr = lshift si by -3 (lo);
dm(canal) = sr0;    // salveaza canalul

call read_extension;    // citeste switch-urile locale
call apply_sw_params;    // aplica parametrii functiei selectate
call update_display;    // actualizeaza afisajul

```

//alegerea functiei

```

ax0 = dm(id_functie);    // citeste ID functie
ay0 = 0;                // AGC are ID 0
ar = ax0 - ay0;          // compara
if eq call do_agc;        // daca e AGC, apeleaza AGC

```

```

ax0 = dm(id_functie);    // reciteste ID
ay0 = 1;                // ALE are ID 1
ar = ax0 - ay0;          // compara
if eq call do_ale;        // daca e ALE, apeleaza ALE

```

```

ax0 = dm(id_functie);    // reciteste ID
ay0 = 2;                // MF are ID 2
ar = ax0 - ay0;          // compara
if eq call do_mf;        // daca e MF, apeleaza MF

```

```

rti;                  // intoarcere din intrerupere dupa procesare

```

cmd_invalida:

```

ax0 = 0;              // comanda invalida
dm(cmd_corect) = ax0;    // salveaza invalid
ax0 = dm(x_in);        // citeste intrarea curenta
dm(y_out) = ax0;        // bypass: iesirea = intrarea
rti;                  // intoarcere

```

/******

* I/O EXTENSIE – citire SW0-SW7 si scriere afisor

*****/

read_extension:

```

si = IO(PORT_IN);      // citeste valoarea switch-urilor
ax0 = si;              // muta in AX0
ay0 = 0x00FF;          // masca pe 8 biti

```

```

ar = ax0 and ay0;      // pastreaza doar byte-ul inferior
dm(sw_value) = ar;     // salveaza valoarea switch-urilor
rts;

```

update_display:

```

ax0 = dm(id_functie);  // citeste functia curenta
si = ax0;              // muta in SI
sr = lshift si by 4 (lo); // id_functie << 4
ay0 = sr0;             // salveaza nibble superior
ax0 = dm(param_id);    // citeste parametrul
ar = ax0 or ay0;       // combina functia si parametrul intr-un byte
dm(displ_value) = ar;  // salveaza pentru diagnostic
si = ar;               // muta in SI
IO(PORT_OUT) = si;     // trimite la afisaj / port de iesire
rts;

```

```

/*****
* PARAMETRI DIN SWITCH-URI
*****/

```

apply_sw_params:

```

ax0 = dm(id_functie);  // functie selectata

ay0 = 0;               // verifica AGC
ar = ax0 - ay0;
if eq jump set_agc_params; // daca e AGC, sare la param AGC

ay0 = 1;               // verifica ALE
ar = ax0 - ay0;
if eq jump set_ale_params; // daca e ALE, sare la param ALE

ay0 = 2;               // verifica MF
ar = ax0 - ay0;
if eq jump set_mf_params; // daca e MF, sare la param MF

rts;                   // altfel nu modifica nimic

```

```

/*****parametrii AGC : SW1-SW0 - REF, SW3-SW2 - mu*****/

```

set_agc_params:

```

/*****REF*****/

```

```

    ax0 = dm(sw_value);          // citeste switch-urile
    ay0 = 0x0003;                // foloseste SW1-SW0 pentru ref
    ar = ax0 and ay0;            // extrage 2 biti
    ay0 = 0;
    af = ar - ay0;
    if eq jump agc_ref_025;      // 00 -> 0.25
    ay0 = 1;
    af = ar - ay0;
    if eq jump agc_ref_050;      // 01 -> 0.50
    ay0 = 2;
    af = ar - ay0;
    if eq jump agc_ref_075;      // 10 -> 0.75
    si = 0.9r;                   // 11 -> 0.90
    jump agc_ref_done;
agc_ref_025:
    si = 0.25r;                  // seteaza referinta 0.25
    jump agc_ref_done;
agc_ref_050:
    si = 0.5r;                   // seteaza referinta 0.50
    jump agc_ref_done;
agc_ref_075:
    si = 0.75r;                  // seteaza referinta 0.75
agc_ref_done:
    dm(agc_ref) = si;            // salveaza referinta

/*****mu*****/
    ax0 = dm(sw_value);          // reciteste switch-urile
    si = ax0;                    // muta in SI
    sr = lshift si by -2 (lo);    //shift dreapta cu 2 pozitii, rezultatul merge in
jumatatea inferioara(lo), adica sr0
    ay0 = 0x0003;                // masca pe 2 biti
    ar = sr0 and ay0;            // extrage selectia mu
    ay0 = 0;
    af = ar - ay0;
    if eq jump agc_mu_001;       // 00 -> 0.01
    ay0 = 1;
    af = ar - ay0;
    if eq jump agc_mu_005;       // 01 -> 0.05
    ay0 = 2;
    af = ar - ay0;
    if eq jump agc_mu_010;       // 10 -> 0.10

```

```

        si = 0.2r;           // 11 -> 0.20
        jump agc_mu_done;
agc_mu_001:
        si = 0.01r;          // seteaza mu=0.01
        jump agc_mu_done;
agc_mu_005:
        si = 0.05r;          // seteaza mu=0.05
        jump agc_mu_done;
agc_mu_010:
        si = 0.1r;           // seteaza mu=0.10
agc_mu_done:
        dm(agc_mu) = si;      // salveaza mu
        rts;

/*****parametrii ALE : SW1-SW0 – mu, SW3-SW2 – lamda *****/
set_ale_params:
/*****mu*****/

        ax0 = dm(sw_value);   // citeste switch-urile
        ay0 = 0x0003;         // foloseste SW1-SW0 pentru mu
        ar = ax0 and ay0;      // extrage 2 biti
        ay0 = 0;
        af = ar - ay0;
        if eq jump ale_mu_0005; // 00 -> 0.005
        ay0 = 1;
        af = ar - ay0;
        if eq jump ale_mu_001;  // 01 -> 0.01
        ay0 = 2;
        af = ar - ay0;
        if eq jump ale_mu_005;  // 10 -> 0.05
        si = 0.1r;             // 11 -> 0.10
        jump ale_mu_done;
ale_mu_0005:
        si = 0.005r;
        jump ale_mu_done;
ale_mu_001:
        si = 0.01r;
        jump ale_mu_done;
ale_mu_005:
        si = 0.05r;
ale_mu_done:

```



```

    dm(ale_mu) = si;          // salveaza mu pentru ALE
/*****lamda*****/
    ax0 = dm(sw_value);      // reciteste switch-urile
    si = ax0;                // muta in SI
    sr = lshift si by -2 (lo); // aduce SW3-SW2 pe pozitia joasa
    ay0 = 0x0003;           // masca
    ar = sr0 and ay0;        // extrage selectia lambda
    ay0 = 0;
    af = ar - ay0;
    if eq jump ale_lam_0005;  // 00 -> 0.0005
    ay0 = 1;
    af = ar - ay0;
    if eq jump ale_lam_001;   // 01 -> 0.001
    ay0 = 2;
    af = ar - ay0;
    if eq jump ale_lam_005;   // 10 -> 0.005
    si = 0.01r;              // 11 -> 0.01
    jump ale_lam_done;
ale_lam_0005:
    si = 0.0005r;
    jump ale_lam_done;
ale_lam_001:
    si = 0.001r;
    jump ale_lam_done;
ale_lam_005:
    si = 0.005r;
ale_lam_done:
    dm(ale_lambda) = si;     // salveaza lambda
    rts;

/*****parametrii MF : SW1-SW0 -> K (1, 1, 2, 3)*****/
set_mf_params:
    ax0 = dm(sw_value);      // citeste switch-urile
    ay0 = 0x0003;           // foloseste SW1-SW0 pentru K
    ar = ax0 and ay0;        // extrage 2 biti
    ay0 = 0;
    af = ar - ay0;
    if eq jump mf_k_1a;      // 00 -> 1
    ay0 = 1;
    af = ar - ay0;
    if eq jump mf_k_1b;      // 01 -> 1

```

```

    ay0 = 2;
    af = ar - ay0;
    if eq jump mf_k_2;      // 10 -> 2
    ax0 = 3;                // 11 -> 3
    jump mf_k_done;
mf_k_1a:
    ax0 = 1;
    jump mf_k_done;
mf_k_1b:
    ax0 = 1;
    jump mf_k_done;
mf_k_2:
    ax0 = 2;
mf_k_done:
    dm(mf_k) = ax0;        // salveaza K
    rts;                   // intoarcere

```

```

/*****
* AGC
*    $y(n) = g(n-1) * x(n)$ 
*    $S = (1/M) * \sum |y(n-k)|, k = 0..M-1$ 
*    $g(n) = g(n-1) + \mu * (REF - S)$ 
*****/

```

```

do_agc:
//configurează pointerul circular spre delay-line-ul AGC - buffer de 4 elemente
unde stau ultimele |y(n)|
    i3 = agc_delay;
    l3 = AGC_M;
    m3 = 1;
    se = -AGC_K;    //pregateste o scalare cu 2^-2 (impartire la 4)

```

*//Citește eșantionul curent x(n) și îl pune în mx0 — registrul sursă al
multiplicatorului MAC*

```

    ar = dm(x_in);
    mx0 = ar;

```

```

    mr = 0;                // reseteaza acumulatorul
    my0 = -1.0r;           // constanta pentru adunare prin inmultire
    ax1 = dm(agc_g_int);    // incarca partea intreaga a castigului
    ay1 = dm(agc_g_frac);    // incarca partea fractionara a castigului

```

```

ar = ax1 - 1;          // verifica daca partea intreaga este 1
if lt jump agc_g_fr;    // daca este 0, sari peste termenul intreg
mr = mr - mx0 * my0 (ss); // daca ax1=1, contribuie cu x(n)
agc_g_fr:
my1 = ay1;             // muta partea fractionara in MY1
mr = mr + mx0 * my1 (rnd); // adauga termenul fractionar => y(n)=g*x

dm(y_out) = mr1;       // salveaza iesirea AGC

//Calculul mediei S
ar = abs mr1;          // |y(n)|
sr = ashift ar (hi);    // scaleaza /4 pentru media mobila
dm(i3, m3) = sr1;      // insereaza in delay line

cntr = AGC_M - 1;      // 3 iteratii in bucla + 1 final
mr = 0, mx0 = dm(i3, m3); // initializeaza suma si primul element
do agc_sum until ce;    // suma bufferului
agc_sum:
mr = mr - mx0 * my0 (ss), mx0 = dm(i3, m3); // adauga elementul curent si
citeste urmatorul
mr = mr - mx0 * my0 (rnd); // adauga ultimul element
if mv sat mr;          // satureaza daca este nevoie (mv – Multiply oVerflow)
dm(agc_S) = mr1;       // salveaza media mobila

// $g(n) = g(n-1) + \mu*(REF - S)$ 
ax0 = dm(agc_ref);     // citeste referinta
ay0 = dm(agc_S);       // citeste media curenta
ar = ax0 - ay0;        // eroarea ref-S
my1 = ar;              // muta eroarea in MY1
mx1 = dm(agc_mu);      // citeste mu
mr = mx1 * my1 (ss);    // calculeaza  $\mu*(ref-S)$ 
ar = mr1 + ay1;        // actualizeaza partea fractionara a castigului

//Gestionarea overflow-ului câștigului
af = pass ar;          // testeaza semnul
if ge jump agc_cont;    // daca este valid, continua normal

af = ax1 - 1;          // altfel verifica daca ax1 era deja 1
if lt jump agc_cor1;    // daca nu, trecem la 1.0
ax1 = 0;               // altfel saturam

```

```

    ay1 = 0x7fff;          // castig maxim reprezentabil
    jump agc_save;         // sari la salvare
agc_cor1:
    ax1 = 1;              // partea intreaga devine 1
    ay1 = 0;              // partea fractionara devine 0
    jump agc_save;        // sari la salvare
agc_cont:
    ay1 = ar;             // actualizeaza partea fractionara normala

agc_save:
    dm(agc_g_int) = ax1;   // salveaza partea intreaga
    dm(agc_g_frac) = ay1;  // salveaza partea fractionara
    rts;                  // intoarcere

/*****
* ALE
*    $y(n) = \sum_{k=0..N-1} h(k) * x(n-k-D)$ 
*    $e(n) = x(n) - y(n)$ 
*    $h(k) = (1 - \mu * \lambda) * h(k) + \mu * x(n-k-D) * e(n)$ 
*****/
do_ale:
    ena ar_sat;           // activează saturarea pe registrul ar — dacă o
                           // adunare/scădere depășește limitele, se saturează

    m1 = 1;               // increment DM
    m5 = 1;               // increment PM
    i2 = ale_input;       // buffer de intrare
    l2 = ALE_N;           // lungime 8
    i4 = ale_input + ALE_DELAY; // pointer decalaj D
    l4 = ALE_N;           // lungime 8
    i3 = ale_fir_d;       // buffer FIR
    l3 = ALE_N;           // lungime 8
    i6 = ale_hh;          // coeficienti
    l6 = ALE_N;           // lungime 8

    ar = dm(x_in);        // citește x(n)
    dm(ale_save) = ar;    // îl salvează pentru eroare  $e(n) = x(n) - y(n)$ 

    dm(i2, m1) = ar;      // scrie x(n) în bufferul de intrare
    ar = dm(i4, m5);      // citește x(n-D)

```

```
dm(i3, m1) = ar;          // scrie x(n-D) in bufferul FIR
```

```
cntr = ALE_N - 1;        // numar de iteratii pentru FIR  
call ale_fir;            // calculeaza iesirea filtrului
```

```
dm(y_out) = mr1;         // salveaza y(n)
```

//calculul erorii

```
ar = dm(ale_save);        // reciteste x(n)  
ay0 = ar;                 // muta in AY0  
ar = ay0 - mr1;           // e(n) = x(n) - y(n)  
mx0 = ar;                 // pune eroarea in MX0
```

```
cntr = ALE_N;             // nr coeficienti de actualizat  
my1 = dm(ale_mu);         // mu  
mx1 = dm(ale_lambda);     // lambda  
m4 = -1;                  // decrement  
m6 = 2;                   // pas pentru scriere  
m3 = -1;                  // corectie pointer  
m7 = 0;                   // acces fara modificare pointer  
call ale_llms;            // actualizeaza coeficientii  
rts;                      // intoarcere
```

ale_fir:

```
mr = 0, mx0 = dm(i3, m1), my0 = pm(i6, m5); // initializare suma si primii  
operanzi
```

```
do ale_sop until ce;      // bucla FIR
```

ale_sop:

```
mr = mr + mx0 * my0 (ss), mx0 = dm(i3, m1), my0 = pm(i6, m5); // MAC +  
preload
```

```
mr = mr + mx0 * my0 (rnd); // ultimul produs + rotunjire
```

```
if mv sat mr;             // saturare daca e cazul
```

```
rts;                      // intoarcere cu rezultatul in MR1
```

ale_llms:

```
mr = mx0 * my1 (rnd), mx0 = dm(i3, m1); // mu*e(n), citeste primul x
```

```
my0 = mr1;                // pastreaza mu*e(n)
```

```
mr = mx0 * my0 (rnd), ay0 = pm(i6, m5); // x*mu*e, citeste primul coeficient
```

```
do ale_adapt until ce;    // bucla pe toti coeficientii
```

```
ar = mr1 + ay0, mx0 = dm(i3, m1), ay0 = pm(i6, m4); // LMS clasic
```

```

mf = mx1 * my1 (rnd), sr1 = pm(i6, m7); // lambda*mu si h(k)
mr = sr1 * mf (rnd);          // h(k)*mu*lambda
ay1 = mr1;                    // salveaza termenul leakage
ar = ar - ay1;                // h = h + mu*x*e - h*mu*lambda
ale_adapt:
    pm(i6, m6) = ar, mr = mx0 * my0 (rnd); // scrie noul coeficient si pregateste
urmatorul
    modify(i3, m3);            // corectie pointer i3
    modify(i6, m4);            // corectie pointer i6
    rts;                       // intoarcere

/*****
* MF
* fereastră: x(n), x(n-1), x(n-2) (ultimele W=3 esantioane)
* sortat: x*(0) <= x*(1) <= x*(2)
* y(n) = x*(K)
*****/
do_mf:
    m1 = 1;                    // increment standard
    m3 = 1;                    // increment standard

    i3 = mf_delay;             // buffer circular pentru fereastră curentă
    l3 = MF_W;                 // lungime 3
    ar = dm(x_in);             // citește esantionul nou
    dm(i3, m3) = ar;           // îl introduce în fereastră

    i0 = mf_delay_sorted;      // pointer la bufferul de sortare
    l0 = 0;                    // buffer liniar
    cntr = MF_W;               // 3 elemente de copiat
    do mf_copy until ce;       // copiază fereastră
    si = dm(i3, m3);           // citește din bufferul circular
mf_copy:
    dm(i0, m3) = si;           // scrie în bufferul liniar

    i2 = mf_delay_sorted;      // baza pentru sortare
    call mf_sort_sel;          // sortează valorile

    i0 = mf_delay_sorted;      // revine la începutul sirului sortat
    l0 = 0;                    // liniar
    ax0 = dm(mf_K);            // încarcă poziția medianei

```

```

m0 = ax0;           // m0 = index median
modify(i0, m0);      // avanseaza la elementul median
ar = dm(i0, m3);     // citeste mediana
dm(y_out) = ar;      // salveaza iesirea MF
rts;                 // intoarcere

```

mf_sort_sel:

```

m3 = 1;             // pas 1
l0 = 0;             // i0 liniar
l1 = 0;             // i1 liniar

```

```

cntr = MF_W;        // bucla exterioara pe toate elementele
do mf_loop_i until ce; // selectie

```

```

ax0 = MF_W;         // valoare constanta W
ay0 = cntr;         // valoarea curenta a contorului
ar = ax0 - ay0;     // i = W - cntr
m0 = ar;            // min = i
m1 = ar;            // retine i
ar = ar + 1;        // i+1
m2 = ar;            // inceput bucla interioara

```

```

ay0 = m1;           // ay0 = i
ar = ax0 - ay0;     // W - i
ar = ar - 1;        // W - i - 1 = nr comparatii

```

```

if eq jump mf_skip_inner; // daca nu mai exista elemente, sari

```

```

cntr = ar;          // seteaza contorul buclei interioare
i0 = i2;            // i0 = baza sirului
modify(i0, m2);     // i0 -> A[i+1]

```

```

do mf_loop_j until ce; // cauta minimul pe restul sirului
ax0 = dm(i0, m3);     // A[j]
i1 = i2;              // i1 = baza
modify(i1, m0);       // i1 -> A[min]
ay0 = dm(i1, m3);     // A[min]
ar = ax0 - ay0;       // compara A[j] cu A[min]
if gt jump mf_skip_min; // daca A[j] > A[min], nu actualiza

```

```

ax0 = MF_W;         // recalculeaza pozitia lui j

```

```

ay0 = cntr;          // din contorul ramas
ar = ax0 - ay0;      // j
m0 = ar;             // min = j

```

mf_skip_min:

mf_loop_j:

```

nop;                // nop de sincronizare

i1 = i2;            // i1 = baza
modify(i1, m1);     // i1 -> A[i]
ax1 = dm(i1, m3);   // ax1 = A[i]
i1 = i2;            // i1 = baza
modify(i1, m0);     // i1 -> A[min]
ay1 = dm(i1, m3);   // ay1 = A[min]
i1 = i2;            // i1 = baza
modify(i1, m1);     // i1 -> A[i]
dm(i1, m3) = ay1;   // A[i] = A[min]
i1 = i2;            // i1 = baza
modify(i1, m0);     // i1 -> A[min]
dm(i1, m3) = ax1;   // A[min] = vechiul A[i]

```

mf_skip_inner:

```

nop;                // nop de sincronizare

```

mf_loop_i:

```

nop;                // nop final bucla exterioara

```

```

m1 = 1;             // readuce m1 la increment 1

```

```

rts;                // intoarcere

```

```

/*****

```

```

* SPORT0 RX

```

```

*****/

```

input_samples:

```

ena sec_reg;        // set secundar de registre

```

```

sr1 = dm(rx_buf + 2); // citeste esantionul audio util din rx buffer

```

```

dm(x_in) = sr1;      // salveaza intrarea curenta

```

```

si = IO(PORT_IN);    // citeste switch-urile

```



```

IO(PORT_OUT) = si;          // le afiseaza direct pe iesire (diagnostic)

ax0 = dm(canal);            // citeste canalul selectat
ay0 = 0;                    // compara cu 0
ar = ax0 - ay0;             // rezultat comparatie
if eq jump out_left;        // daca 0, proceseaza pe canal stang

ax0 = dm(y_out);            // altfel foloseste iesirea procesata
dm(tx_buf + 2) = ax0;        // scrie procesatul pe canal drept
sr1 = dm(rx_buf + 1);        // citeste canalul stang din intrare
dm(tx_buf + 1) = sr1;        // lasa stangul in bypass
jump out_done;              // sari la final

out_left:
    ax0 = dm(y_out);         // iesirea procesata
    dm(tx_buf + 1) = ax0;     // scrie procesatul pe canal stang
    sr1 = dm(rx_buf + 2);     // citeste canalul drept original
    dm(tx_buf + 2) = sr1;     // lasa dreptul in bypass

out_done:
    ax0 = dm(Prog_Flag_Data); // citeste toti pinii portului PF7-0
    ay0 = 0x00FF;             // masca pe 8 biti
    ar = ax0 and ay0;         // pastreaza doar byte-ul inferior (PF7-0)
    dm(PF_input) = ar;        // salveaza valoarea citita in variabila PF_input
    rti;

/*****
****
* SPORT0 TX

*****/

next_cmd:
    ena sec_reg;              // foloseste setul secundar
    ax0 = dm(i3, m1);         // citeste urmatoarea comanda init
    dm(tx_buf) = ax0;         // o pune in tx buffer
    ax0 = i3;                 // copie pointerul curent
    ay0 = init_cmds;          // adresa de baza
    ar = ax0 - ay0;           // calculeaza offsetul
    if gt rti;                // daca mai sunt comenzi, iese
    ax0 = 0xaf00;              // altfel trimite comanda finala
    dm(tx_buf) = ax0;         // scrie in tx buffer

```

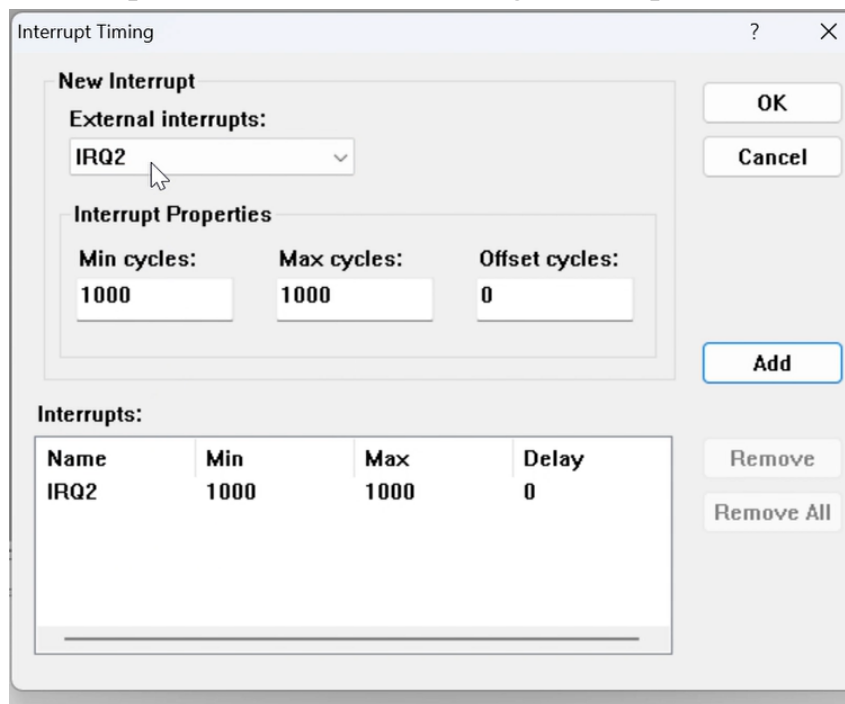
```

ax0 = 0;           // init terminat
dm(stat_flag) = ax0; // reseteaza flagul
rti;              // intoarcere

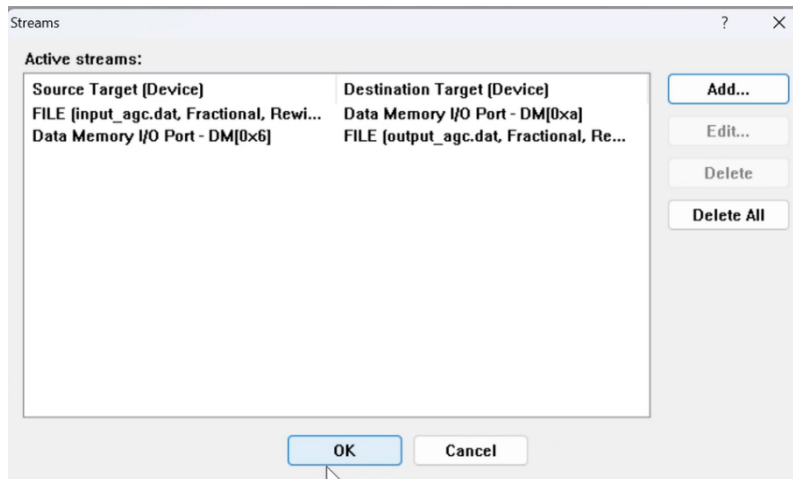
```

10. Simulări în VisualDSP++

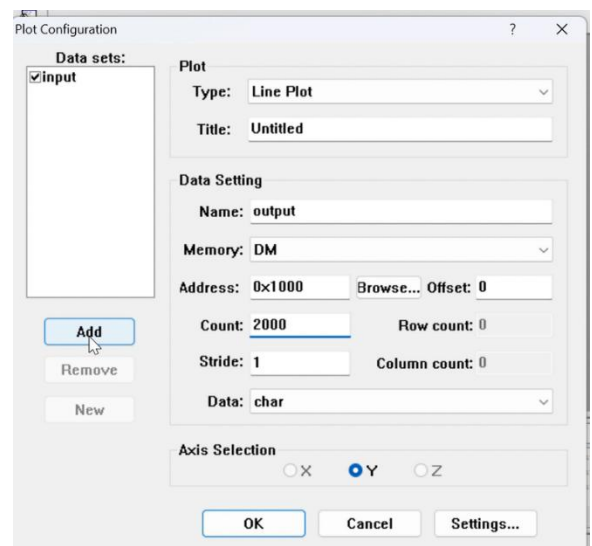
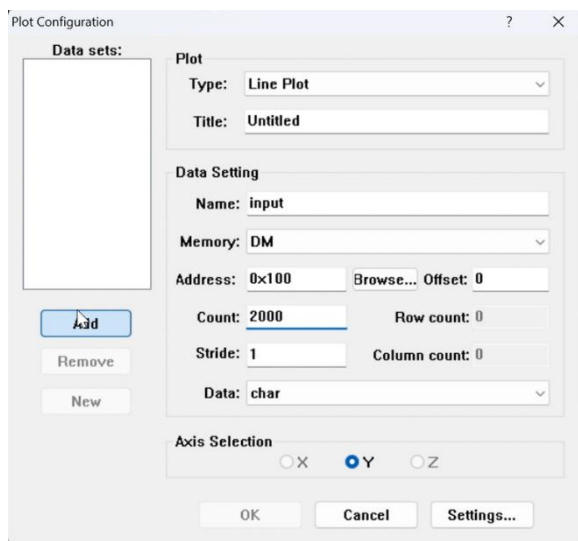
Am configurat o întrerupere externă pe linia IRQ2. Am stabilit valoarea minimă și valoarea maximă pentru cicluri la 1000, asigurând o periodicitate constantă



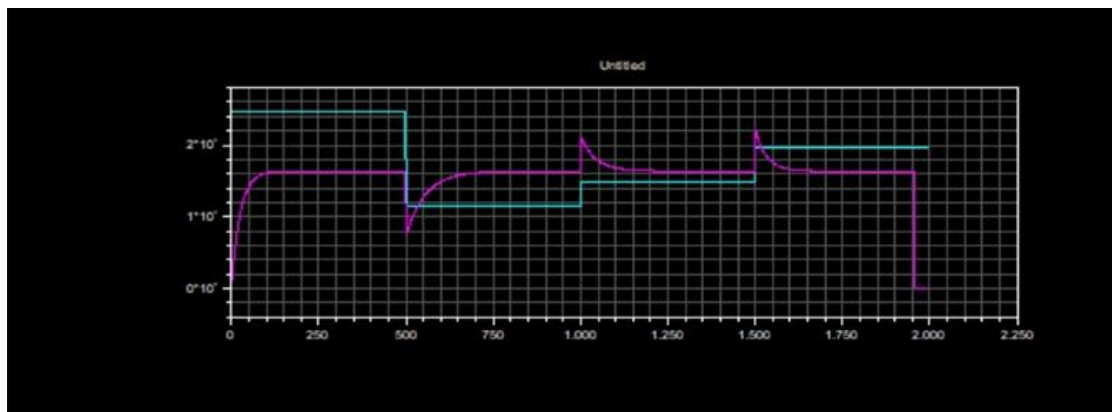
Am realizat configurarea pentru transmisia și recepția datelor utilizând Streams din meniul VisualDSP++. Citirea semnalului de intrare se face din poziția rx_buf + 2, aceasta reprezentând eșantionul efectiv utilizat de algoritmul de procesare. În mod similar, rezultatul obținut după prelucrare este scris în tx_buf + 2. Astfel, rx_buf a fost folosit ca sursă de date pentru funcțiile de procesare, iar tx_buf ca destinație pentru semnalul rezultat. Această configurare asigură un transfer eficient de date între memorie și fișier.



Pentru a vizualiza datele, vom selecta o adresă de la care să începem afișarea valorilor din meniul View -> Debug Windows -> Plot.

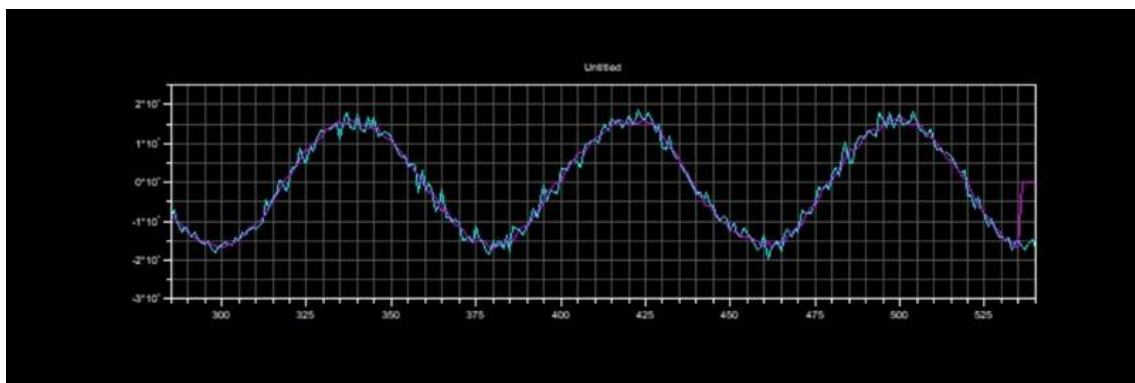


Automatic Gain Control – AGC



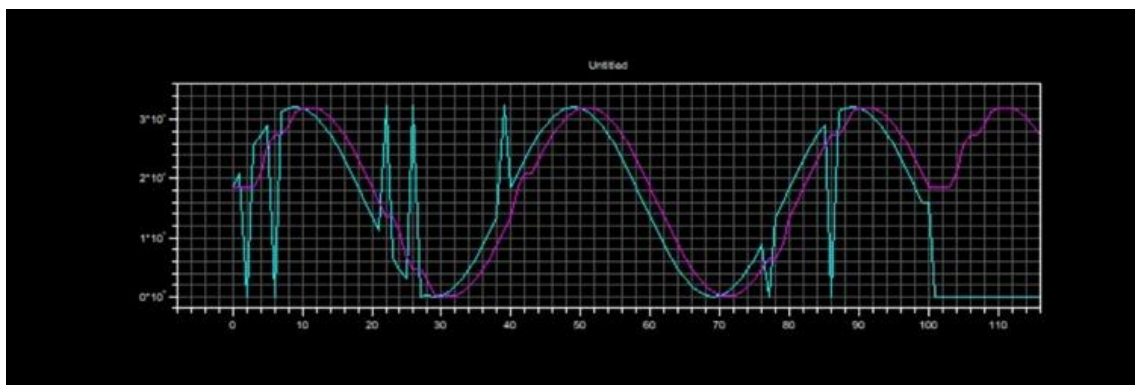
albastru – intrarea ; *roz* – ieșirea

Adaptive Line Enhancer – ALE



albastru – intrarea ; *roz* – ieșirea

Median Filter – MF



albastru – intrarea ; *roz* – ieșirea